Open GIS Consortium

35 Main Street, Suite 5
Wayland, MA 01778
Telephone: +1-508-655-5858
Facsimile: +1-508-655-2237

Editor:
Telephone: +1-703-830-6516
Facsimile: +1-703-830-7096
ckottman@opengis.org

The OpenGIS™ Abstract Specification
Topic 2: Spatial Reference Systems

Version 4

OpenGIS™ Project Document Number 99-102r1.doc

NOTICE

The information contained in this document is subject to change without notice.

The material in this document details an Open GIS Consortium (OGC) specification in accordance with the license and notice set forth on this page. This document does not represent a commitment to implement any portion of this specification in any companies' products.

While the information in this publication is beleived to be accurate, the Open GIS Consortium makes no warranty of any kind with regard to this material including but not limited to the implied warranties of merchantability and fitness for a particular purpose. The Open GIS Consortium shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance or use of this material. The information contained in this document is subject to change without notice.

The Open GIS Consortium is and shall at all times be the sole entity that may authorize developers, suppliers and sellers of computer software to use certification marks, trademarks, or other special designations to indicate compliance with these materials.

Revision History

Date	Description
22 March 1999	Carry forward 98-102r1 as 99-102; use new document template following guidance of change proposal 99-010 w/ friendly amendments to remove Section 1 boilerplate from individual topic volumes, approved 9 February 1999; update copyrights for 1999; use Annex A of change proposal 98-072, approved at the 9 February TC plenary meeting as placeholder for Section 3; incorporate change proposal 99-018r2, approved at the 9 February 1999 TC plenary, into Section 1.2, Section 1.3, Section 2 and Section 3.
21 May 1999	Replace Section 3 (Abstract Model) and add Sections 7 and 8 (appendices) with content from change proposal 99-005r3 approved at the April 1999 TC plenary.

This page is intentionally left blank.

Table of Contents

Table of Figures

1. Introduction

1.1. The Abstract Specification

The purpose of the Abstract Specification is to create and document a conceptual model sufficient enough to allow for the creation of Implementation Specifications. The Abstract Specification consists of two models derived from the Syntropy object analysis and design methodology [1].

The first and simpler model is called the Essential Model and its purpose is to establish the conceptual linkage of the software or system design to the real world. The Essential Model is a description of how the world works (or should work).

The second model, the meat of the Abstract Specification, is the Abstract Model that defines the eventual software system in an implementation neutral manner. The Abstract Model is a description of how software should work. The Abstract Model represents a compromise between the paradigms of the intended target implementation environments.

The Abstract Specification is organized into separate topic volumes in order to manage the complexity of the subject matter and to assist parallel development of work items by different Working Groups of the OGC Technical Committee. The topics are, in reality, dependent upon one another— each one begging to be written first. *Each topic must be read in the context of the entire Abstract Specification.*

The topic volumes are not all written at the same level of detail. Some are mature, and are the basis for Requests For Proposal (RFP). Others are immature, and require additional specification before RFPs can be issued. The level of maturity of a topic reflects the level of understanding and discussion occurring within the Technical Committee. Refer to the OGC Technical Committee Policies and Procedures [2] and Technology Development Process [3] documents for more information on the OGC OpenGIS™ standards development process.

Refer to Topic Volume 0: Abstract Specification Overview [4] for an introduction to all of the topic volumes comprising the Abstract Specification and for editorial guidance, rules and etiquette for authors (and readers) of OGC specifications.

1.2. Introduction to Spatial Reference Systems

1.2.1. Background on the Notion of a Feature

The quantum of geographic information is the feature. A feature object (in software) corresponds to a real world or abstract entity. Attributes of (either contained in or associated to) this feature object describe measurable or describable phenomena about this entity. Unlike a data structure description, the geodata model objects derive their semantics and valid use or analysis from the corresponding real world entities' meaning.

1.2.2. Background on the Notion of a Reference System

The Open Geodata Specification Model (OGM) begins by building literally from the ground up. Using the concept of reference systems, attribute primitives such as geographically registered geometry are tied to the real world. Feature objects are then decorated in a controlled manner with attributes.

1.2.3. Background on Fundamental Data Model Functionalities

The OGM does not depend on specifics of the implementation environment, but does assume the existence of some basic type or object functionality. This functionality, if not supplied directly by the computing environment, will have to be implemented by the OGC programmer. Some of this functionality includes:

1. Classes of collectors: collectors based on unordered sets and ordered lists (or arrays) are needed. Other collectors such as bags (multi-sets) may be useful in implementations but these can be constructed from sets of simple tuples. Syntactic differences such as between directly addressable arrays and sequentially addressable lists are part of the implementation specification.

2. Tuples or structured typed heterogeneous aggregates: combinations of mixed types will be needed. These are the equivalent of relational row types, C programming structures, or aggregated objects in object oriented programming languages (OOPL).

3. Schema and Dynamic type system: the ability to import, export, read, write and store structure information and to build types (classes) dynamically at run-time is probably a necessity.

1.2.4. Background on the Notion of Location: Place and Time

Place and time in the geodata model do not correspond to software entities. They are part of the boundary between the software and the 'real world.' Place is a measurable piece of the real world. Time is a point, an interval or collection of points and intervals in what we perceive as the time continuum. Time and place can be measured and surveyed, and their coordinates in a particular spatial, temporal reference system can be derived. Because the approach in the model can unify place and time, we will use 'location' to mean both.

The following UML model diagram shows how location is related to the software entities described later in this chapter. Spatial temporal reference systems, and coordinate geometries are defined later. Aggregation (represented by the diamond on the aggregate side) means dependence of existence. The diagram implies that a semantically complete and correct coordinate geometry cannot exist without the real world location that it references and the reference system that it uses to reference it. The dashed line means that the reference system is an attribute of the relationship between the two objects. Here it implies that the location and its coordinate geometry software object are related through use of the reference system.

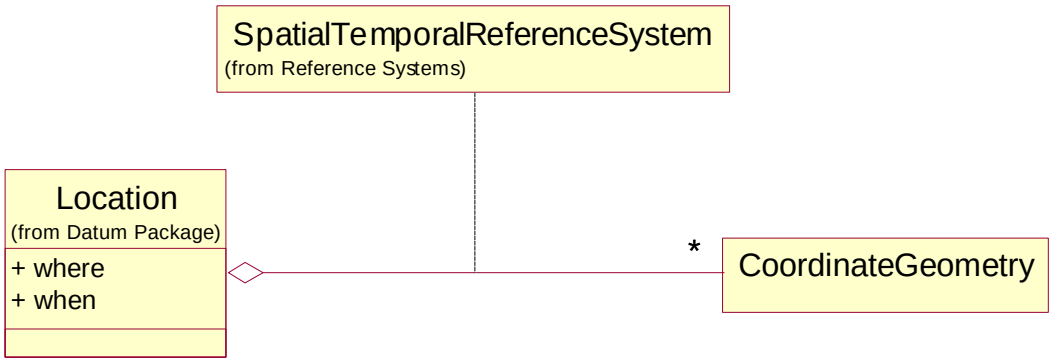

Figure 1-1. Location and Geometric Representation

1.2.5. General Notion of a Reference System

The use of reference system in the previous section is actually a specific example of a more general modeling phenomena. In the Open Geodata Model, digital objects representing entities are associated to attribute values to imply that the corresponding entity in the real world has a description that parallels this attribution. This implies that to maintain the semantic attachment to its entity, the object should have a mechanism to associate it attribute values (digital objects which are member variables or functions for the feature object) to the implied real world description.

A reference system is a means of assigning values to a location, time or other descriptive quantity or quality. In the most general sense, a reference system can be thought of as a scale of measurement. This can include ordinal and cardinal scales, or descriptive vocabularies. The scales can be discrete or continuous, linear or cyclic. Normally spatial and temporal reference systems are restricted to cardinal, continuous scales with values in a real vector space.

A spatial reference system is a function which associates locations in space to geometries of coordinate tuples in a mathematical space, usually a real valued coordinate vector space, and conversely associates coordinate values and geometries to locations in the real world.

A temporal reference system is a function which associates time to a coordinate (usually 1 dimensional points and intervals) and conversely associates coordinate geometries to real world time.

A spatial temporal reference system is an aggregation of a spatial system and a temporal system that it uses to associate coordinate geometries to locations in space and time. Normally, the aggregation uses orthogonal coordinates to represent space and time, but this is not necessarily the case in more complex, relativistic environments.

Attribute reference systems perform a similar role for numeric attributes (such as money, temperature, pressure and other physical measures). Some attribute reference systems may be as simple as assigning a unit of measure to an attribute, while others can be quite complex, such as money systems that include temporally continuous rates of exchange between currencies.

Normally, the function used in a reference system for a numeric attribute is topologically bicontinuous (invertible with a continuous inverse), and, depending on the circumstances, either one of the two functions may be of primary concern.

One may speak of several reference systems within one statement. When this occurs, one may use variables and the resolution operator "::" to track which system is in force for a particular variable definition. For example R::Geometry would refer to a geometry in the spatial reference system R.

The general setting of a Reference System is shown in the following figure.

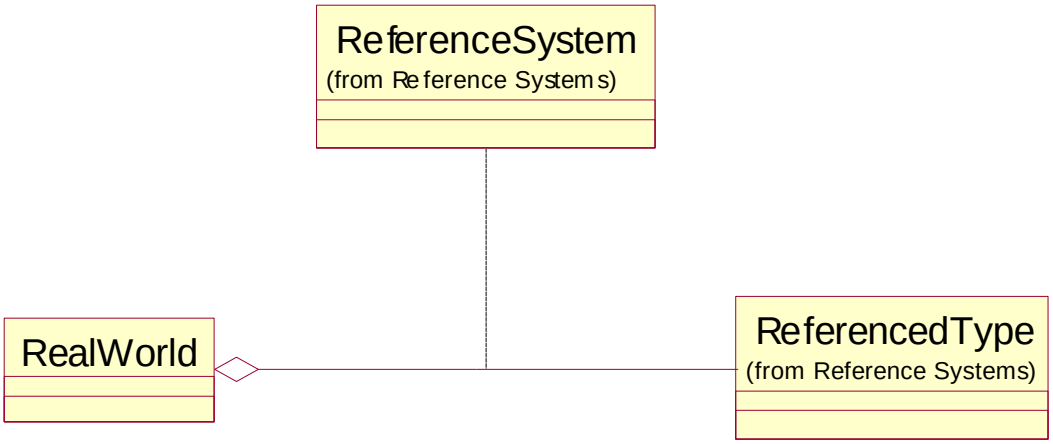

Figure 1-1. Reference System Map Types to Real World Phenomena

The referenced type in Figure 1-1 can be any OpenGIS type that can be used to describe a feature. The reference system is associated to one referenced type and will map this type to a real world meaning.

1.3. The Status of this Topic

At the Cambridge, UK, OGC meetings, August 11-14, 1997, Simple Feature Implementation Specifications were accepted as OGC baseline, in accordance with the OGC RFP consensus process. These specifications are available at http://www.opengis.org/members/spec_rev.htm

The Implementation Specifications add implementation detail to the Abstract Model presented in this Topic Volume. Specifically, the EPSG database of geodetic information is used broadly.

Significant additions were made to the Spatial Reference Systems specification in 1997. See especially Section 2 references [5], [9], and [10].

In particular, an Annex, Section 6, titled "An introduction to coordinate system geodesy", has been added to assist the understanding of this Topic.

1.4. References for Section 1

[1] Cook, Steve, and John Daniels, Designing Objects Systems: Object-Oriented Modeling with Syntropy, Prentice Hall, New York, 1994, xx + 389 pp.

[2] Open GIS Consortium, 1997. OGC Technical Committee Policies and Procedures, Wayland, Massachusetts. Available via the WWW as <http://www.opengis.org/techno/development.htm>.

[3] Open GIS Consortium, 1997. The OGC Technical Committee Technology Development Process, Wayland, Massachusetts. Available via the WWW as <http://www.opengis.org/techno/development.htm>.

[4] Open GIS Consortium, 1999. Topic 0, Abstract Specification Overview, Wayland, Massachusetts. Available via the WWW as <http://www.opengis.org/techno/specs.htm>.

2. The Essential Model for Spatial Reference Systems

2.1. Introduction

For the user, spatial coordinates are abstractions of locations in space. As far as the user of GIS is concerned, spatial coordinates are names for locations that enable operations such as finding the distance between two points expressed with coordinates, or finding the intersection between two areas whose boundaries have been expressed with coordinates.

2.1.1. Why Spatial Coordinates are Important in GIS

Spatial Coordinates are needed and used in GIS in many ways. Here are some examples:

1. A ship determines its location (relative to nearby things, or in an abstract way) by plotting GPS coordinates on a chart

2. A parcel of land has its boundary expressed in terms of a sequence of points, each represented by its coordinates (such as feet north and west of a Section corner). The section corner itself has coordinates in a state plane.

3. An airplane determines its location using latitude, longitude and elevation, that is, its (lat., long., and h coordinates) in a ellipsoidal reference system.

In addition to state plane coordinates and ellipsoidal coordinates, there are:

- local coordinates
- geocentric coordinates
- Cartesian coordinates
- vertical coordinates
- projected coordinates
- photograph coordinates
- stereo-object-space coordinates
- display coordinates
- military grid coordinates
- and many others, all of which have good reasons to exist.

2.1.2. What are Coordinates?

Coordinates are tuples that take numeric values in each position, or ordinate. (The singular term "coordinate" sometimes refers to the tuple, and sometimes to a single ordinate in the tuple. The plural term "coordinates" sometimes refers to a single tuple, and sometimes to a set of tuples.) An instance of coordinates is an ordered list of numeric "ordinates", that are the values of the tuple in its respective positions.

More generally, an instance of coordinates is an element of a coordinate space, where a coordinate space may be taken to be (all or a portion of) a mathematical finite-dimensional vector space over the real numbers. The mathematical vector space (or a portion of it), is called a coordinate space, or space of coordinates.

We might say that a two-dimensional coordinate space is the set {(x,y)| x and y are real numbers}, (or a subset of it), and a three-dimensional coordinate space is the set {(x,y,z)| x, y, and z are positive real numbers} (or a subset of it), and so on.

This section will show that coordinates are used to designate, or "name" positions in space in particular reference systems.

2.1.3. Spatial Coordinates

Spatial coordinates are coordinates with spatial semantics. These semantics can get complex, and it is the goal of this Essential Model to clarify the semantics. At the heart of spatial coordinate semantics is the relationship between coordinates and place. That is, spatial coordinates establish a relationship between the places (or "spots" or "atomic pieces" on or near the earth's surface -- note that we are avoiding the term "point" because this term has a special meaning in Geometry) and coordinates in a coordinate space.

There are two ideas here. The first idea relates a bag of coordinates to a bag of corresponding places. The second idea relates a singular place to a singular tuple. Of course, the first idea follows from the second.

This topic volume, and the OGC Abstract Specification in general, usually assumes that places are on or near the earth's surface. However, other spatial regions of discourse can be needed, some with similar properties. For example, the places could be on or near the surface of another planet or of a moon orbiting a planet.

2.1.4. Alternative to Coordinates

Spatial Coordinate Systems are one way to accomplish spatial referencing. In general, we call a system of spatial referencing a Spatial Reference System. There are two kinds of Spatial Reference Systems: those that spatially reference by coordinates, and those that reference by identifiers. This topic volume addresses only the first kind.

2.1.5. GIS Scenarios that Involve Spatial Coordinates

There are two fundamental operations in GIS that involve coordinates. Neither is directly implemented in software, but instead are human activities.

Assume that we are given an instance of coordinates from a particular space of spatial coordinates. (Note that we have just been given a set of semantics, as well as a tuple of numbers. These semantics relate a point in a coordinate space to point on or near the earth's surface.) The first fundamental operation is finding a place on or near the surface of the earth semantically related to that coordinate. This operation may be called "***locate***." To accomplish this operation, one may need hiking boots, surveying tools, monuments, a GPS receiver, or other devices.

Now assume that we are given a well-defined place on or near the surface of the earth, and further assume that a "sufficiently rich" set of spatial coordinate semantics is in effect. The second fundamental operation is determining an instance of coordinates semantically related to the well-defined place. This operation is called "***survey***." To accomplish the survey operation, one may need the same or similar tools as for the locate operation.

Locate is an operation on spatial coordinate semantics taking one argument (an instance of coordinates) that returns a place on or near the surface of the earth.

Survey is an operation on spatial coordinate semantics taking one argument (a place) and returning an instance of coordinates.

A place on or near the surface of the earth is called "well-defined" if:

1. It can be taken to be a point (as the tip of a steeple, or the center of an iron pin, a building corner, or the etched cross-hair on a survey marker.) (Actually, we sometimes allow linear and areal features to be well-defined places that are associated semantically to collections of coordinates, but this is not important to the current discussion.)

2. It may be repeatedly visited (that is, the intended point is mutually understood by all professional persons visiting its neighborhood.)

3. It doesn't move. (Or, it moves at geologic speed, where the motion [an earthquake, or slow deformation of rock] can be accounted for by deleting one set of semantics and replacing with another [that accommodates any shift caused by the earthquake].)

2.1.6. "Sufficiently Rich" Spatial Coordinate Semantics

For now, we will say that the semantics of a spatial coordinate system are sufficiently rich if different persons (who are well trained and practiced in such activities) can reliably and repeatably (that is, get the same results -- within the error tolerance) exercise the locate and survey methods. The next few sections are devoted to establishing the characteristics of a sufficiently rich set of spatial coordinate semantics.

2.2. Spatial Coordinates Semantics

In spatial coordinates, there are three important objects:

1. The "earth surface" object. This is an object with lots of "places", and may include places that are somewhat above or below the actual surface. The oceanic part of this surface is often modeled by the "geoid," a gravity-related mathematical surface (often developed with spherical harmonics to be globally defined.) Operations often exercised on the earth surface object include those performed with a transit, tape measure, gravimeter, thermometer, barometer, laser range finder, star sighting instrument, camera, and so on. Operations on this object include measurements, observations, placing stakes (making arbitrary places well defined), and finding shortest paths, perimeters, areas, and so forth. Time is important, too, because the earth is moving relative to the sun and stars, (and for other reasons) but we postpone the discussion of time. It is important to think of the earth surface as Euclid would have: it is a rigid metric geometry in which points, lines, length, surfaces, cones, ellipses, area, volume, and so forth, have intrinsic meaning.

2. A mathematical object. This object is always a set of (mathematical) points, and is often taken to be the points on (or near) a particular oblate ellipsoid, (which is defined as an ellipse rotated about its minor axis). In any event, the points in the mathematical object are always represented by coordinates. The coordinates may be those in a plane containing the mathematical object, or in all or part of a three-dimensional vector space (where the points are identified with vectors). It is usually useful to think of the mathematical object as embedded in a vector space of appropriate dimension. Operations on this object include exposing axes and units, plotting (finding points with given) coordinate instances, exposing coordinates of selected points, and executing vector arithmetic.

3. The relationship object. This object establishes a relationship between the earth surface and the mathematical object, and thereby determines the semantics of the spatial coordinates. The relationship can be one-to-one or many-to-one. This object is also called a Coordinate Reference System.

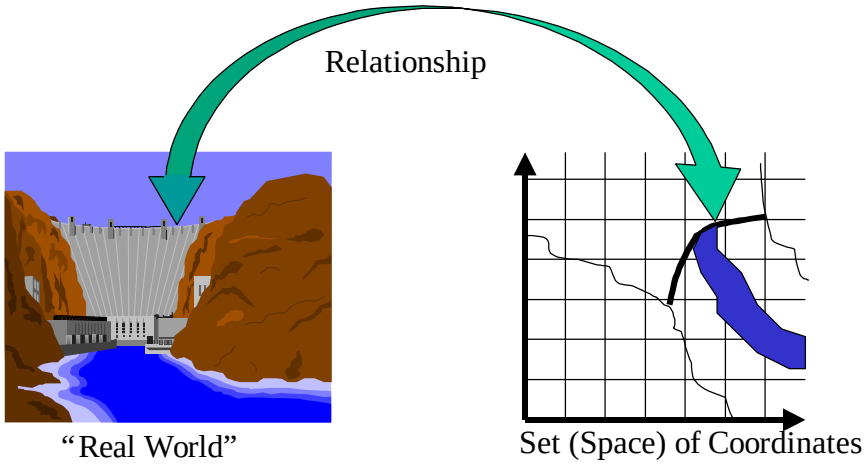

Figure 2-1. The Relationship between Locations and Coordinates

2.2.1. Coordinate Reference System Names

In day-to-day commerce, the whole of spatial coordinate semantics are communicated by the "name" of the Coordinate Reference System. Coordinate Reference System names are similar to this (simplified) example:

> Coordinates are northing and easting in the Lambert projection, with tangent at 45 degrees, with origin at 100 degrees west at the equator, and using the Clarke 1866 ellipsoid and datum.

2.2.2. Types of Coordinate Reference Systems

There are a relatively small number of commonly used Coordinate Reference System types. The spatial coordinate semantics are peculiar to each one, so they must be explained one-by-one. Refer also to Section 6 (Appendix A. An introduction to coordinate system geodesy). In this Essential Model, we briefly introduce the following common spatial coordinate reference systems:

1. 2-D ellipsoidal. The origin of this type of system is typically assumed to be located at or near the Earth's geocenter. An ellipsoid, with a "geocentric" origin and a size and shape that approximates the size and shape of the earth, may be used to represent a geodetic reference system. The 2-D refers to the fact that we are only interested in using this system to derive two horizontal position components (latitude and longitude). Note that three orthogonal coordinate axes must still be defined in order to determine the latitude and longitude coordinates. A third (ellipsoid height) component can also be derived but we will defer discussion of heights to the paragraph on linear systems. Horizontal and height position components can come together in a compound coordinate reference system.

2. 3-D Cartesian. The origin may be located at or near the geocenter (i.e., geocentric), or at or near a point on the earth's surface (i.e., topocentric).

3. Polar (also known as spherical). The origin may be located at or near the geocenter (i.e., geocentric) or, more typically (as in surveying with a total station or theodolite), at or near a point on the earth's surface (i.e., topocentric).

4. Projected. A special 2-D Cartesian system in which the origin is topocentric and only "horizontal" components are considered.

5. Vertical. A 1-D system that can be used in connection with heights. Heights may be referenced to a mathematical surface, such as the ellipsoid, or a gravity-related surface, such as the geoid. For height systems, the coordinate axis is usually oriented in the direction normal to the particular reference surface, and the origin is defined at the point of intersection of the coordinate axis and the reference surface. Heights have no meaning without an association to a place, whether the place is described by coordinates or by another attribute such as a name.

6. Compound. An aggregated or composite system that combines a "horizontal" system, such as 2-D ellipsoidal, with an independent linear/height system.

7. Other taxonomies of coordinate systems are often encountered, and one of these is presented in Section 2.6.

2.3. Monuments and Datums

It is common for the relationship between the earth surface object and the mathematical object to be related in terms of "monuments." A ***monument*** is two things:

1. a well known place (point)

2. an assigned instance of coordinates.

(It is possible that the same well known place is used with two different coordinate instances. In this case, we will treat the situation as two monuments, not one.)

A monument may be associated with additional information, such as a North vector, a local vertical vector, or a vector to another monument.

A collection of monuments (there may just be one monument in the collection) is often used to establish the relationship between the earth surface object and the mathematical object in a spatial coordinate system. Think of a rigid framework of steel beams that connect the monuments. The

mathematical object (usually an ellipsoid) is "rigidly moved" about the earth surface object (about the rigid framework of steel beams) until its coordinates at the monuments match the assigned spatial coordinates at each monument. There may be other adjustments as well, such as making an ellipsoid axis point north, making local vertical vectors normal to the ellipsoid, and so on. The adjustments may involve a least-squares or another "best fit" algorithm.

When the mathematical object is "pinned" to the earth's surface, the relationship between places and coordinates becomes more clear. The collection of monuments is called a datum, and the relationship between places and coordinates is called a Coordinate Reference System. Each spatial coordinate instance becomes co-located with a place, and each place becomes co-located with a spatial coordinate instance.

2.4. Domain of Validity

Each Coordinate Reference System is associated with a datum, and the datum can be local or global. When the datum is local (perhaps restricted to a country or continent), the corresponding Coordinate Reference System is useful only in the neighborhood of its datum. For now, a domain of validity is assumed to be defined for each Coordinate Reference System. For places within the domain of validity, the relationship between places and coordinates can be trusted. Outside its domain of validity, a Coordinate Reference System should not be used.

Every Coordinate Reference System should support an operation that exposes the domain of validity of that system Exceptions should be raised if an operation is invoked on a coordinate outside that domain. A more advanced environment would expose an appropriate domain of validity for each of several given tolerances.

2.5. Datum Transformations and Accuracy

Places may be located in more than one datum. If the coordinates of a place or places can be represented in two datums, then a mathematical transformation describing the relationship between the coordinates on the two datums can be defined. This mathematical transformation is often referred to as a datum transformation, and is the subject of a later section.

The accuracy of the transformation will depend on factors such as the accuracy of the coordinates (in their respective datums), the number of points, and the transformation technique used to establish the transformation.

2.6. Another Taxonomy of Coordinate Reference Systems

There are many Coordinate Reference Systems in use, and many taxonomies of them are possible. This section presents a slightly different list that that of Section 2.2.2. Refer also to Section 6 (Appendix A. An introduction to coordinate system geodesy). At this Essential Model level, one taxonomy is not preferred over another, and the introduction to each one is kept brief.

2.6.1. 2-D Ellipsoidal with Ellipsoidal Height, h.

In an ellipsoidal compound coordinate reference system, the mathematical object is an oblate ellipsoid. Points on the earth's surface are "above" (outside) or "below" (inside) the ellipsoid. The distance from a point on the earth's surface (distance is measured along the shortest distance to the ellipsoid, that is, along a normal to the ellipsoid) is called "height", h, or "ellipsoidal height. A special point is identified as establishing the prime meridian, from which longitude is measured. The latitude of a point is defined as the angle subtended between the ellipsoid's equatorial plane and a line from the point drawn perpendicular to the ellipsoid surface. (Strictly, this is the geodetic latitude. Other types of latitude exist.)

2.6.2. Ellipsoidal with Gravity-related Height

Here, the mathematical object is an ellipsoid, as in 2.6.1, and latitude and longitude coordinates are exactly the same as in that section. Height, H, however is measured normal to the surface of the geoid or another gravity-related equipotential surface. This compound system is sometimes referred to as the 2.5-D reference system.

2.6.3. Geocentric Cartesian

Geocentric Cartesian coordinates can be derived from ellipsoidal coordinates.

The idea is to use the same 3 coordinate axes defined for a particular geodetic/ellipsoidal reference system to define a three-dimensional Cartesian coordinate system that "embeds" the ellipsoid.

There is a straightforward conversion between (latitude, longitude, ellipsoid height) and (x,y,z). Of course the conversions are different between geodetic height and gravity based height. Conversion from ellipsoidal with gravity related height is done in two steps, conversion to ellipsoidal with ellipsoid height, then conversion to geocentric Cartesian.

2.6.4. Polar

Any conversion of ellipsoidal coordinates, or Cartesian coordinates to polar coordinates (r, ϕ, θ) = (radius, vertical/elevation angle, horizontal/azimuth angle) yields polar coordinates. The forward and reverse conversions are well known.

2.6.5. Cylindrical

Any conversion of ellipsoidal coordinates, or Cartesian coordinates to cylindrical coordinates (r, ϕ, z) = (radius, , horizontal/azimuth angle, elevation (distance) above the horizontal plane) yields cylindrical coordinates. The forward and reverse conversions are well known.

2.6.6. Vertical

Vertical or height coordinates are often related to a different datum from horizontal ones. Vertical coordinates can provide an ordinate at each point of the ellipsoid, and this ordinate can be appended to the 2-dimensional ellipsoid coordinates to create ordered triples. Alternately, vertical coordinates can provide an ordinate (a 1-tuple) at each point of the geoid. In any case, vertical (elevation or depth) ordinates are a function of some pre-existing 2-dimensional coordinates.

In summary, vertical coordinates can be thought of as a coverage on an ellipsoid (or on a geoid). The coverage assigns to each point of the ellipsoid (or geoid) an ordinate (which is semantically interpreted as a height above or below some reference surface (often the same or another ellipsoid or geoid.) The "heights" here are often taken to be vectors whose directions need to be stated explicitly. They may be gravity-related, ellipsoid-related, or taken to be parallel to a given (z-axis) vector.

2.6.7. Compound Coordinate Reference Systems

A Compound Coordinate References System may combine axes from two or more Coordinate Reference Systems. For example there may be axes from one or more 2- or 3-dimensional Coordinate Reference Systems, and additional axes for various depth and elevation datums.

2.6.8. Projected

Projected coordinates are the two-dimensional Cartesian coordinates typically found on maps and computer displays. The Cartesian axes are often called "paper coordinates" or "display coordinates." The conversions from a three-dimensional curvilinear coordinate system (whether ellipsoidal or spherical) to projected coordinates may be assumed to be well known. Examples of projected coordinate systems are: Lambert, Mercator, and transverse Mercator. Conversions to, and conversions between, projected spatial coordinate systems often do not preserve distances, areas and angles.

2.6.9. Linear

A linear system is a coordinate system restricted to points along a curved (or straight) line, where the coordinate values are related to distance along the curve from specified origins. A linear coordinate system can be used to form a vertical coordinate reference system as described above.

2.7. Common Notation

The following notation is common:

(φ, λ, h_e)	ellipsoidal with ellipsoidal height, where φ = latitude, λ = longitude
(φ, λ, h_g)	ellipsoidal with gravity-related height
(x_g, y_g, z_g)	geocentric Cartesian
(x, y, z)	Cartesian
(ρ, θ) or (ρ, θ, χ)	polar (where θ = azimuth, χ = elevation)
$v = f(\varphi, \lambda)$	vertical
(φ, λ, v)	compound

(E, N)	projected
(x,y) or (x,y,z) = $f_s(t)$	linearly referenced where s = segment, t = parameter

2.8. Other Topics Important to Spatial Reference Systems

2.8.1. Discussion of Photogrammetry

There are many interesting coordinate systems in photogrammetry: image coordinates, epipolar axes, stereo model coordinates, local tangent planes, perspectives, and more. A detailed discussion is found elsewhere.

Note: the reader interested in Photogrammetry is referred to Topic Volumes 7, 15 and 16.

2.9. Coordinate Conversions and Transformations

The term "conversion" is used when the relationship between the input and output coordinate systems can be expressed as a closed mathematical relationship, and the calculation taking input coordinates to output coordinates can be executed to any desired error tolerance, no matter how fine. The "fineness" is merely a function of the number of series terms used in the development of the conversion algorithm, and whether or not the conversion method requires iteration and the convergence criteria used to control the iterations. The important point is that a conversion operation does not change the datum or ellipsoid upon which the coordinates are based.

The term "transformation" is used when there is a lower limit, L, such that the act of changing of coordinates unavoidably introduces error of magnitude at least L. This term is commonly used in connection with datum transformations. Datum transformations are based on empirical measurements of position (observations) that are subject to errors. These errors are quantifiable and can be propagated through to the transformation parameters that are established to define the relationship between coordinates on two datums.

2.9.1. Introduction to Coordinate and Datum Transformations

Suppose you have a bag of red_datum points and a bag of blue_datum points.

- If you work with just the blue bag you can ignore the datum entirely as long as you label the points blue.
- If you work with both bags you need to be able to grab a coordinate transformation, but you don't need to know anything about the datum other than that one is labeled red, the other blue, and that a transformation between red and blue is available.
- If you work with GPS and a blue map, you punch "blue" into the GPS and put a blue tag on the coordinates it gives you.

The only time you need to think about what "blue" actually means is if you work with satellites or are establishing a datum, which you probably aren't.

2.9.2. Coordinate Transformations

Coordinate Transformations are functions. The domain of a Coordinate Transformation function is a subset of the domain of validity of a Coordinate Reference System, CRS1. The range is a subset of the domain of validity of another Coordinate Reference System, CRS2. If a Coordinate Transformation maps one instance of coordinates in CRS1 into a second instance of coordinates in CRS2, then the place located at the first coordinates in CRS1 and the place located at the second coordinates in CRS2 are the same.

The terms "source" and "target" differentiate the two instances of coordinates.

In general, Coordinate Transformations do not preserve area, length, or direction, and the Jacobian of the Transformation is a useful tool. [Reference a textbook on Calculus of Several Variables.]

Cartographic projections are examples of Coordinate Transformations. Such transformations usually have a 3-dimensional domain, and a 2-dimensional range.

2.9.3. Elementary Transformations

Transformations from certain Coordinate Reference Systems to certain others are mathematically straight-forward, and can be easily accomplished in a "single step." These are called elementary transformations. Often, when transforming from CRS1 to CRS2, there will be a chain of transformations, starting at CRS1 and ending at CRS2, where each link in the chain is an elementary transformation. Traversing such a chain accomplishes a "Concatenated Transformation", and the accomplishment of each link in the chain is called a Transformation Step

2.10. Position Accuracy

The numerical values of coordinates almost always have limited accuracies. If the accuracy were truly unknown, the numerical value would have little practical value. If the accuracy is approximately known, that accuracy should be communicated from the data producer to the data user, so that the data user can properly interpret and use the numerical value. Recording of position accuracy information is therefore required by the OpenGIS Abstract Specification, Topic 1: Feature Geometry, in Figure 2-1 and Section 2.2.1.3. Topic volume 9 specifies in considerable detail the accuracy data mentioned only briefly elsewhere in the Abstract Specification.

2.10.1. Output Coordinates Accuracy Data

In addition to producing coordinates, coordinate transformation services should output data describing the position accuracy of these output coordinates. Position accuracy output data is not useful in all cases, but should be available whenever it is needed. This accuracy data should take the form of position error statistical estimates. Absolute error estimates should be available for single points, relative to the defined coordinate SRS. Relative error estimates could be made available for pairs of points. These error estimates should be in the form of covariance matrices, as defined and specified in Abstract Specification Topic 9: Quality.

2.10.2. Output Accuracy Determination

Error estimates for coordinates output from a transformation should represent the combination of all significant error sources (or error components). In general, three types of error sources need to be considered: input coordinate errors, transformation parameter errors, and transformation computation errors.

The input coordinates used by the transformation will often contain errors, which propagate to the output coordinates. Error estimates for the input coordinates can be propagated to the resulting output error estimates by using the partial derivatives of the individual output coordinates with respect to the individual input coordinates.

The transformation parameters used will often contain errors, which propagate to the output coordinates. Error estimates for the transformation parameters can be propagated to the resulting output error estimates by using the partial derivatives of the individual output coordinates with respect to the individual transformation parameters.

The transformation computations will sometimes introduce significant errors, at several steps in the computation, which propagate to the output coordinates. Typical computation errors can be propagated to the resulting output error estimates by using the partial derivatives of the individual output coordinates with respect to the internal quantities where errors are committed.

2.10.3. Input Coordinates Accuracy Data

In order to produce output coordinates accuracy data, coordinate transformation services will usually need similar accuracy data for the input coordinates. This input accuracy data should reflect the estimated errors in determining the coordinates. In some cases, the proper image coordinates error estimates will be zero.

2.10.4. Coordinate Transformation Accuracy Data

To produce output coordinates accuracy data, coordinate transformation services will also generally need accuracy data for the transformation performed. This transformation accuracy data might be for the transformation as a whole, or for the various parameters used in the transformation. A coordinate transformation between different ground SRSs will sometimes have significant errors. However, ground coordinate “conversions” will have zero error estimates, except for computation errors.

Computation errors might or might not be negligible even when double precision floating point arithmetic is used in coordinate transformations. For example, the 48 bit precision of a double precision floating point number applied to a longitude of about 180 degrees corresponds to an error of about 8 micrometers. In performing coordinate transformation, several such errors can be committed. Although very rare, some ground coordinates could have similar size errors from all other sources.

Data about the inherent accuracy of a coordinate transformation could be recorded as metadata for that transformation. Data about the computation error in an implementation of a coordinate transformation could be either internal data of the implementation or metadata about that implementation.

2.11. References For Section 2

Editor's Note: These references are included here for informational purposes only. They are not specifically referenced in the text of this section.

[1] POSC (Petrotechnical Open Software Consortium), Epicentre Model V 3.1, http://www.petroconsultants.com/epsgweb/epsg.htm, April, 1997.

[2] Ritter, Niles, and Michael Ruth, GeoTIFF Specification V 1.0, ftp://www-mipl.jpl.nasa.gov/pub/geotiff/specification.html.

[3] Ruth, Michael, and Niles Ritter, GeoTIFF Format for Exchange of Raster Data, Presented GIS/LIS, November, 1995.

[4] Ritter, Niles, and Michael Ruth, The GeoTIFF Data Interchange Standard for Raster Geographic Images, Int'l Hourn. Remote sensing, 1997, vol 18, No. 7, p. 1637-1647.

[5] Snyder, John, 1987, Map Projections, A Working Manual, USGS Professional Paper 1395, ix + 381 pp.

[6] DMA, 1990: Datums, Ellipsoids, Grids, and Grid Reference Systems (UNCLASSIFIED), DMA Technical Manual 8358.1.

3. The Abstract Model for Spatial Reference Systems

3.1. Overview of the Object Model

3.1.1. Packages

The object model (OM) is organized as follows:

It contains five major sections each equivalent to a package of the OM that has been developed to date. The five class diagrams of each package are Coordinate System, Datum, Coordinate Transform, Datum Transformation and Coordinate Conversion. The document also contains package of classes from other topic volumes of the OGC Abstract Specification (Image Coordinate Transformation Service and Geometry) and models that have been proposed and accepted for addition to the Abstract Specification, (e.g., the Unit of Measure and Basic Data Types). This model also includes classes from an Accuracy Package currently being proposed for inclusion in the Abstract Specification.

At the beginning of each section is a brief description of the class diagram that follows the description and then the text specification (i.e., a brief description of the each class and each class's attributes and operations generated to date).

In each section, a given class may appear in more than one class diagram. They have been included in an attempt to show completeness or to add clarity. When they have been added to a diagram in this manner the graphical figure for the class has been shaded in gray. Their textual description has NOT been included in the section where they appear as a shaded figure, but in the section more fully describing the class. An example of this is the "Datum" class, which appears in the Coordinate System Class Diagram, but the Datum Class is in another package (i.e., Datum) where the emphasis has been placed on the Datum Class and its associated classes.

The figure below show the major **packages** of the Coordinate Transformation (CT) object model listing the major classes of each one. Packages are a mechanism supported by the Unified Modeling Language (UML) that allows the grouping of similar entities of class diagrams. It has been provided to help a reader gain insight into the groupings of classes in the CT object model.

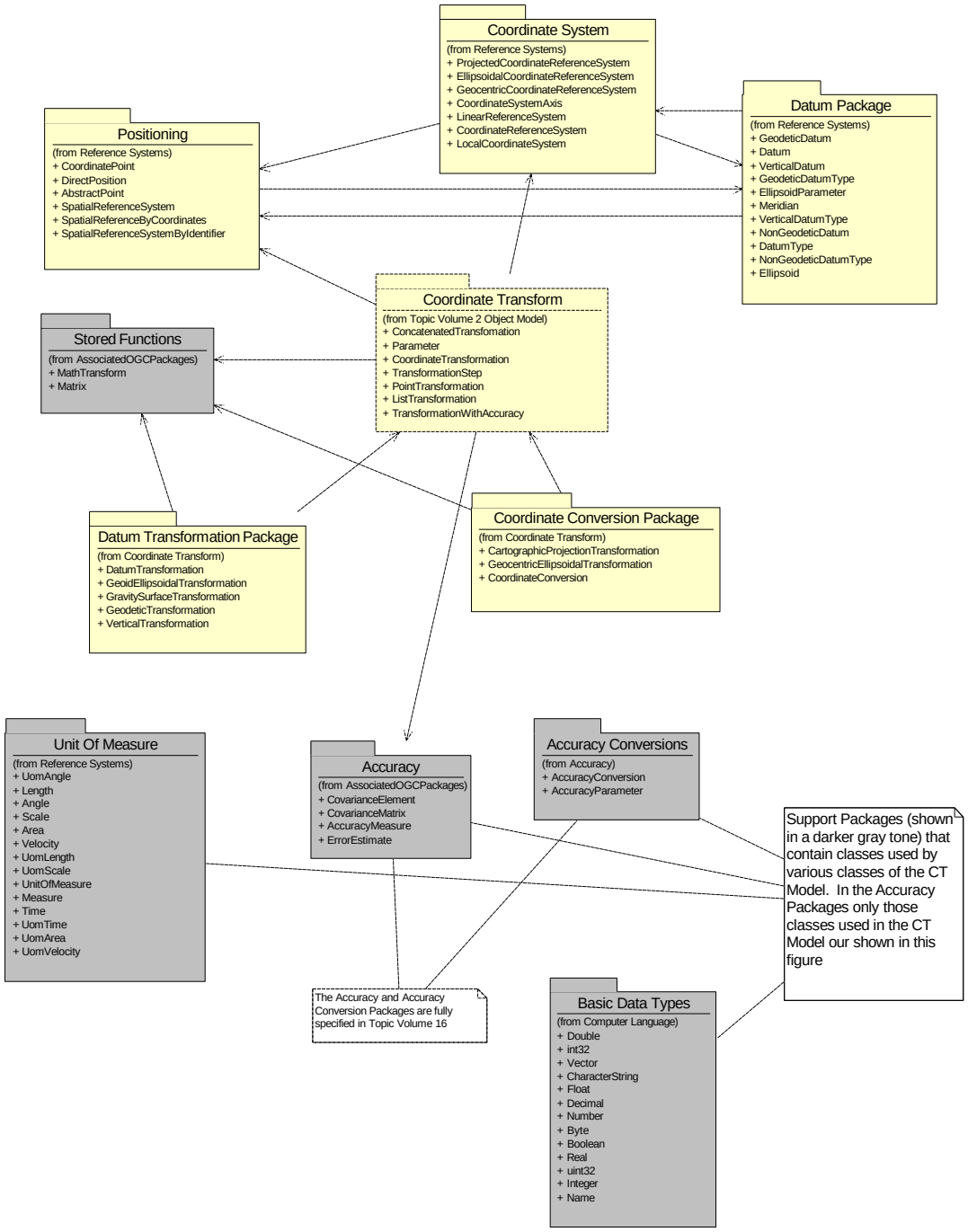

Figure 3-1. Major Class Packages and Supporting Packages of The CT Object Model.

3.1.2. Overview of the Packages Shown

The packages shown above can be broken down into three major categories. The packages shown at the top of Figure 3-1 contain mostly the type of geospatial information that would either support or would be manipulated/used by a coordinate transformation service (CTS). The "Positioning

Package" contains the basic classes (e.g., a CoordinatePoint) that would be manipulated in performing a CT. It also contains the abstract classes generated to show the scope of the object model through the use of a taxonomy shown in Figure 3-1.

The Coordinate System Package and the Datum Package contain many "data type" classes either used to support a CTS or are the "target" of a such a Service.

The three packages situated in the middle of Figure 3-1 contains the actual interfaces and classes that the ad hoc group feels would comprise a CTS. The main package of this set of three is the Transformation Package with the Datum Transformation and Coordinate Conversion Packages containing more specialized classes to do either a transformation or a conversion.

The packages shown at the bottom level of Figure 3-1 (in a darker gray tone in the figure) are supporting packages for the CT Service. The "Unit Of Measure" Package contains classes that would be used in support of a CT Service. Classes such as "Measure", "Length" and "Area" are currently included in this Package. The "Basic Data Types" Package is a subpackage of the "Computer Language" Package and it contains the basic types available in almost all computer languages. The "Stored Functions" Package is also shown in Figure 3-1 as a package whose classes are currently accessed by classes and interfaces of the CT object model.

Figure 3-1 is included here in an attempt to show how this OM "fits" within a given taxonomy of Spatial Referencing Systems. This OM has been developed to concentrate on providing a CT Service for CRSs and transformations on Datums. The taxonomy presented aligns with two ISO TC/211 work items (5). Currently draft international standards are being developed by various groups of TC/211 that concentrate on "Part 11: Spatial Referencing By Coordinates", and "Part 12: Spatial Referencing by Geographical Identifier". The table below provides definitions for those classes shown above that are not discussed elsewhere in this document. The "ImageCoordinateSystem", -Class has been shown in an attempt to illustrate the concept of a local coordinate system. In this case, it is assumed that the image or photo has not been processed (i.e., an image transformation has not been performed on it to align/register it with a given CRS).

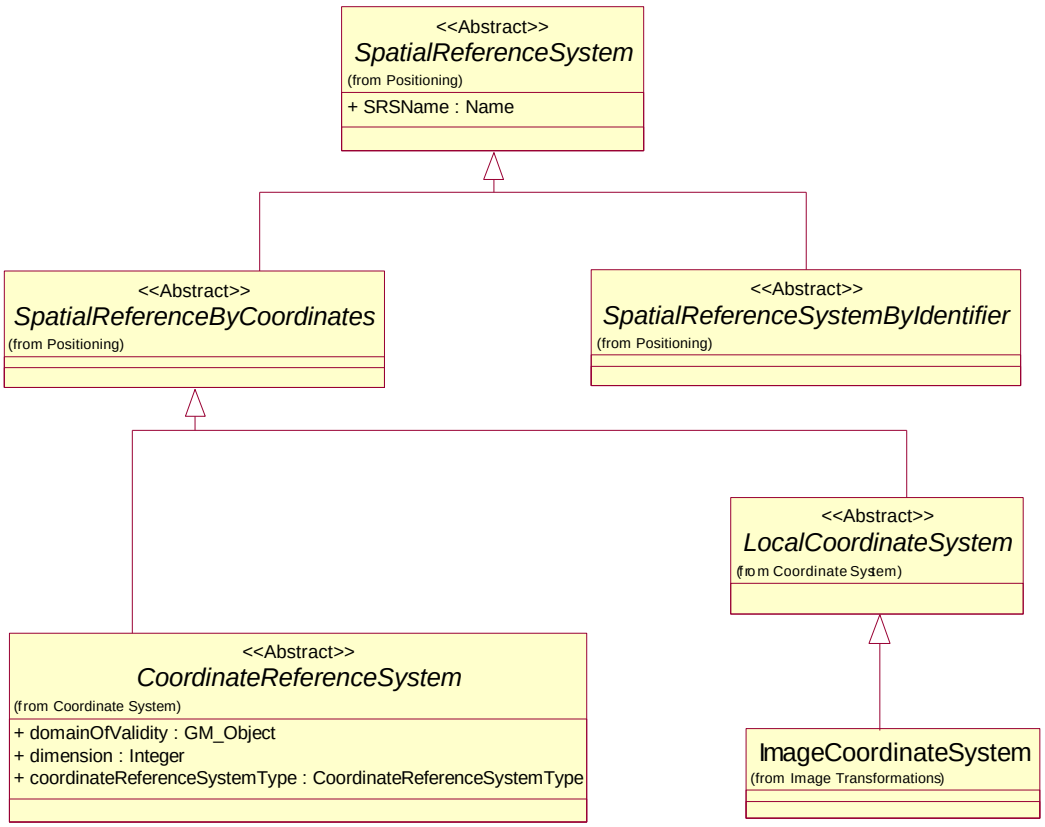

Figure 3-1. Classes of a Spatial Reference System Taxonomy

Class Name	Documentation
LocalCoordinateSystem	Coordinate Systems defined locally and not tied to a geoid or an ellipsoid, and therefore not transformable into a CRSs without more information. Examples are local engineering surveys and architectural drawings.
ImageCoordinateSystem	Defines the image coordinate reference system for a specific version of a specific image, and (probably) associates this image coordinate system with that version of that image.
SpatialReferenceByIdentifier	From ISO 15046-12 : Spatial referencing by geographic identifiers Spatial references using geographic identifiers that are not based explicitly on coordinates, but on association with a location defined by a geographic feature or features. The association of the position to the feature may be: a) containment, where the position is within the geographic feature, for example in a country; b) based on local measurements, where the position is defined relative to a fixed point or points in the geographic feature or features, for example at a given distance along a street from a junction with another street; and

	c) loosely related, where the position has a fuzzy relationship with the geographic feature or features, for example adjacent to a building or between two buildings.

Table 3-1. Definitions of Classes Shown in Figure 3-1.

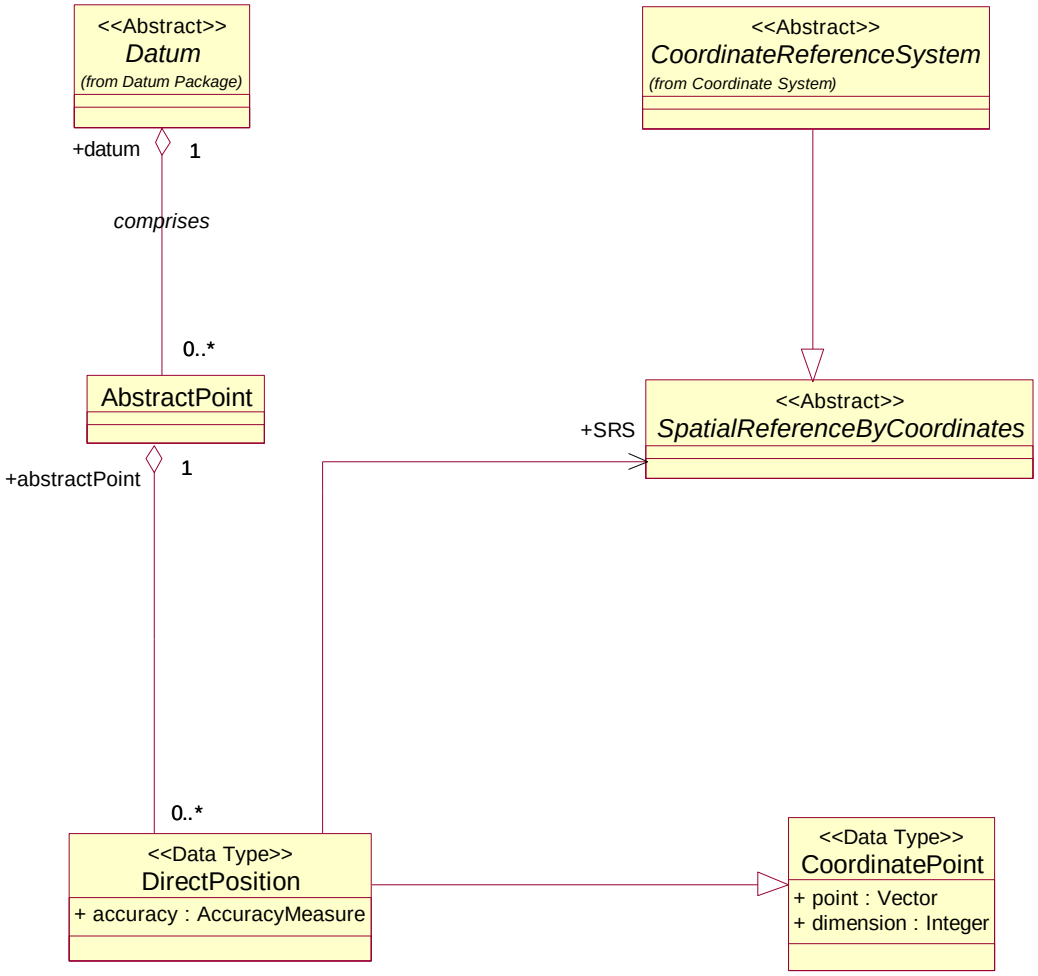

Figure 3-2. Base Classes Being Manipulated in A Coordinate Transformation Service.

The classes shown above are the base classes being used and/or manipulated in a CTS. The four classes listed below have been placed in the Positioning Package of the CT object model. The text specification for the AbstractPoint, the DirectPosition and the CoordinatePoint and the SpatialReferenceByCoordinates are listed below. The other two classes are defined elsewhere in this document.

3.1.3. Class name: AbstractPoint

Package the Class belongs to: Positioning

Documentation:

An equivalence class of direct positions that are related by conversions (i.e., lossless transformations).

Hierarchy:

none

Associations:

<abstractPoint> : DirectPosition in association: ,<unnamed>

<datum> : Datum in association <comprises.>

3.1.4. Class name: DirectPosition

Package the Class belongs to: Positioning

Documentation:

A single set of ordinates within a fixed spatial reference system.

Hierarchy:

Superclasses: CoordinatePoint

Associations:

<abstractPoint> : DirectPosition in association: ,<unnamed>

<SRS> : SpatialReferenceByCoordinates in association: ,<unnamed>

Public Interfaces:

Attributes: accuracy

Attributes:

accuracy : AccuracyMeasure

A measure of the accuracy of one quantity or a related set of quantities.

3.1.5. Class name: CoordinatePoint

Package the Class belongs to: Coordinate System

Documentation:

A sequence of N numbers designating the position of a point in N-dimensional space.

Hierarchy:

Superclasses:

Associations:

<no rolename> : SpatialReferenceByCoordinates in association: ,<unnamed>

Public Interfaces:

Attributes: point

Dimension

Attributes:

point : Vector

An ordered list of numbers used to represent a point in a Cartesian coordinate system.

Each number in the list is referred to as an ordinate. The list together as a whole is a coordinate point (coordinated set of ordinates).

dimension : Integer

The number of ordinates of a CoordinatePoint

3.1.6. Class name: SpatialReferenceByCoordinates

Package the Class belongs to: Positioning

Documentation:

A mechanism for uniquely assigning coordinates (n-tuples of numbers) to positions on the surface of the earth. Given that this assignment is unique, it may be considered an association.

Stereotype: abstract

Hierarchy:

Superclasses: SpatialReferenceSystem

3.2. Coordinate System Package

This Package shows the relationships of a Coordinate Reference System (CRS) and includes a list of CRS subtypes. Note that the subtypes of the CRS are somewhat immature in their development in the model at this time (i.e., the ad hoc modeling group has not tried to specify all the attributes and operations for these classes). Only four subtypes are shown for the CRS, because these were considered to major CRSs most commonly used. The code list for CRS Types does include an "Other" type for any implementor to expand on the subtypes supported.

The CTWG made the decision at the April 1999 TC meeting to leave the multiplicity between the CRS and the Datum Classes as unspecified. This has been done to allow potential implementors of the CT object model the flexibility to define what the multiplicity should be in their particular implementation. This makes the OM more flexible. For example, a latitude, longitude, height system can be described (using this model) as either two CRS, one with a geodetic datum and one with a vertical datum or as a single CRS with two datums.

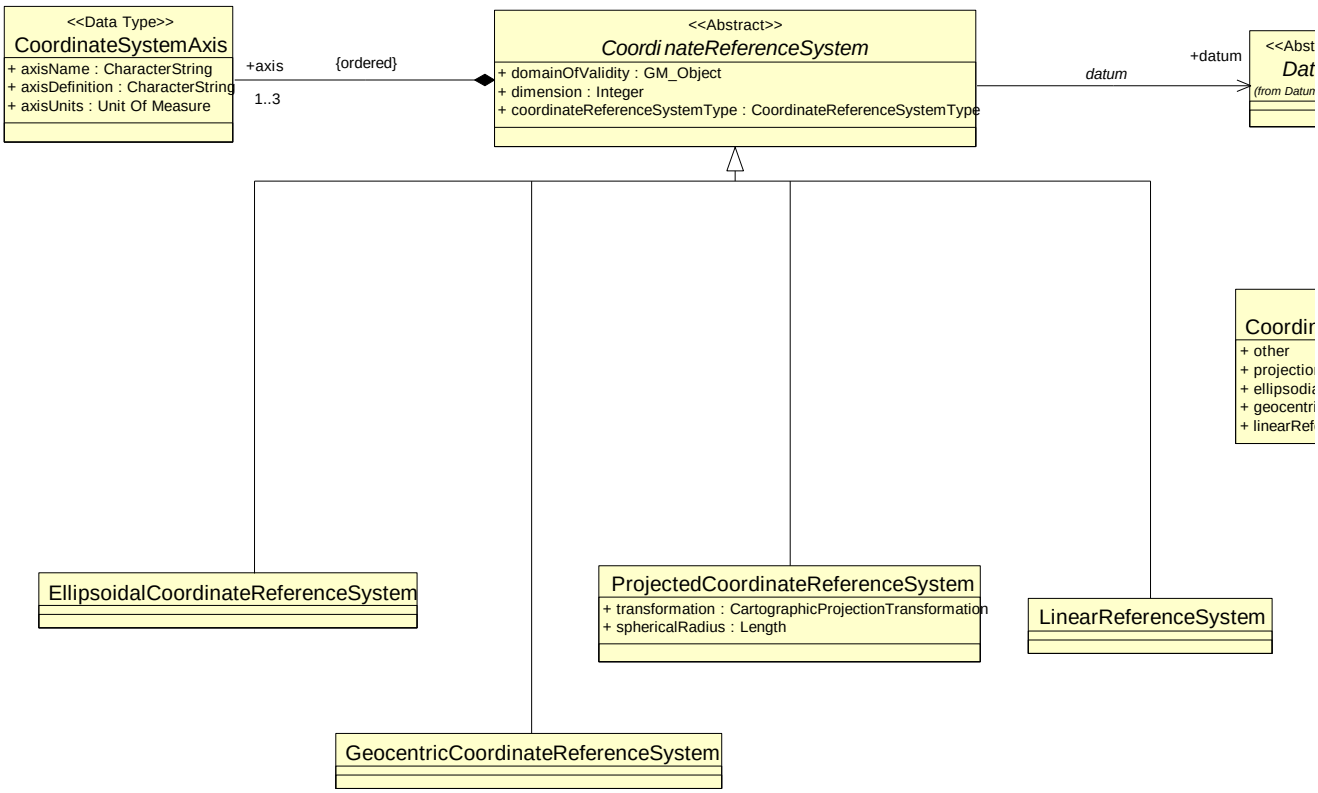

Figure 3-1. Main Class Diagram for the Coordinate System Package.

3.2.1. Class name: CoordinateReferenceSystem

Package the Class belongs to: Coordinate System

Documentation:

This class is a spatial reference system; a mapping between coordinates and earth positions that uses a datum and rules of geometry.

The default coordinate reference system supported is 3-Dimensional.

Stereotype: Abstract

Hierarchy:

Superclasses: SpatialReferenceByCoordinates

Associations:

<theDatum> : Datum in association <Datum>

<axis> : CoordinateSystemAxis in association <unnamed>

Public Interfaces:

Attributes: coordinateReferenceSystemName

domainOfValidity

dimension

coordinateSystemType

Attributes:

coordinateReferenceSystemName : CharacterString

The name of the coordinate reference system (CRS).

domainOfValidity : GM_Object

The domainOfValidity attribute is a geometry in a given CRS that supports the GM_Object interface. This geometry is intended to define the geographical region where this CRS is appropriate to use, or where it historically has been used (in the case of locally mandated systems). The CRS of this object may be in this or some standard coordinate reference system.

dimension : Integer

The number of axes of the CRS.

coordinateSystemType : CoordinateReferenceSystemType

The type of coordinate reference systems currently supported by the CT object model. See the code list class in the main diagram of the Coordinate System Package for CRSs currently included in the model.

3.2.2. Class name: CoordinateSystemAxis

Package the Class belongs to: Coordinate System

Documentation:

The set of axes used to describe a given coordinate system.

Hierarchy:

Superclasses: none

Associations:

<axis> : CoordinateReferenceSystem in association <unnamed>

Public Interfaces:

Attributes: axisName

axisDefinition

axisUnits

Attributes:

axisName : CharacterString

The name of the axis (e.g., x,y,z; latitude or longitude)

axisDefinition : CharacterString

The defining characteristics of an axis. It may include the direction of the axis. Valid examples include: north, south, east, west, up, down.

axisUnits : Unit Of Measure

The unit of measure of the coordinate system axis.

3.2.3. Class name: GeocentricCoordinateReferenceSystem

Package the Class belongs to: Coordinate System

Documentation:

A coordinate reference system that has three perpendicular axes defined in reference to the ellipsoid. The origin of this CRS type is the center of the ellipsoid. The center of the ellipsoid is located at or near the center of the earth.

Hierarchy:

Superclasses: CoordinateReferenceSystem

3.2.4. Class name: LinearReferenceSystem

Package the Class belongs to: Coordinate System

Documentation:

A coordinate reference system restricted to a single curvilinear feature based upon distance measurements along the curve. For example, reference systems based on road mile markers.

Hierarchy:

Superclasses: CoordinateReferenceSystem

3.2.5. Class name: EllipsoidalCoordinateReferenceSystem

Package the Class belongs to: Coordinate System

Documentation:

A coordinate reference system in which position is specified by latitude, longitude and (in the 3D case) ellipsoidal height.

The ellipsoid height (i.e., the distance of a point from the ellipsoid along the perpendicular to the ellipsoid at a point, positive if it is upward (outward from the ellipsoid center) and negative if downward (depth measure).

Hierarchy:

Superclasses: CoordinateReferenceSystem

Associations:

derived : ProjectedCoordinateReferenceSystem in association: sourceEllipsoidalCRS

3.2.6. Class name: ProjectedCoordinateReferenceSystem

Package the Class belongs to: Coordinate System

Documentation:

The ProjectedCoordinateReferenceSystem Class is a 2-D Cartesian coordinate system.

This class is intended to encompass all 2-D coordinate reference systems. Usually, the projection preserves certain relationships such as distance, area and/or angle.

Hierarchy:

Superclasses: CoordinateReferenceSystem

Public Interfaces:

Attributes:

transformation : CartographicProjectionTransformation

The projection transformation that may be used in doing the transformation between the ProjectedCRS and the source ellipsoidal CRS.

sphericalRadius: Length

This attribute holds the value of the spherical radius used in generating the projected CRS.

If the value of this attribute is zero or negative then the ellipsoidal form of the mapping equations is in use, which utilizes the equatorial radius and eccentricity values from the Ellipsoid class.

If the value of this attribute is greater than zero, then the spherical form of the mapping equations is in use, which ignores equatorial radius and eccentricity values from the Ellipsoid class, and instead utilizes the radius stored in sphericalRadius attribute.

3.2.7. Class name: CoordinateReferenceSystemType

Package the Class belongs to: Coordinate System

Documentation:

A CodeList of coordinate reference systems currently proposed as being supported by this object model. The "type" of the code list attributes has been left to the implementor to determine (i.e., whether alphanumeric or just numeric codes are used).

Stereotype: CodeList

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: other

projection

ellipsoidal

geocentric

linearReference

Attributes:

other

Included for the model to be extensible to other CRSs.

projection

A 2-D Cartesian coordinate reference system.

ellipsoidal

A coordinate reference system in which position is specified by latitude, longitude and (in the 3D case) ellipsoidal height.

geocentric

A coordinate reference system that has three perpendicular axes defined in reference to the ellipsoid.

linearReference

A coordinate system restricted to a single curvilinear feature based upon distance measurements along the curve.

3.3. Datum Package

This package shows the relationships of a datum, ellipsoid, and meridian. The package includes three subtypes of datum and includes a list of datum types and ellipsoid parameters.

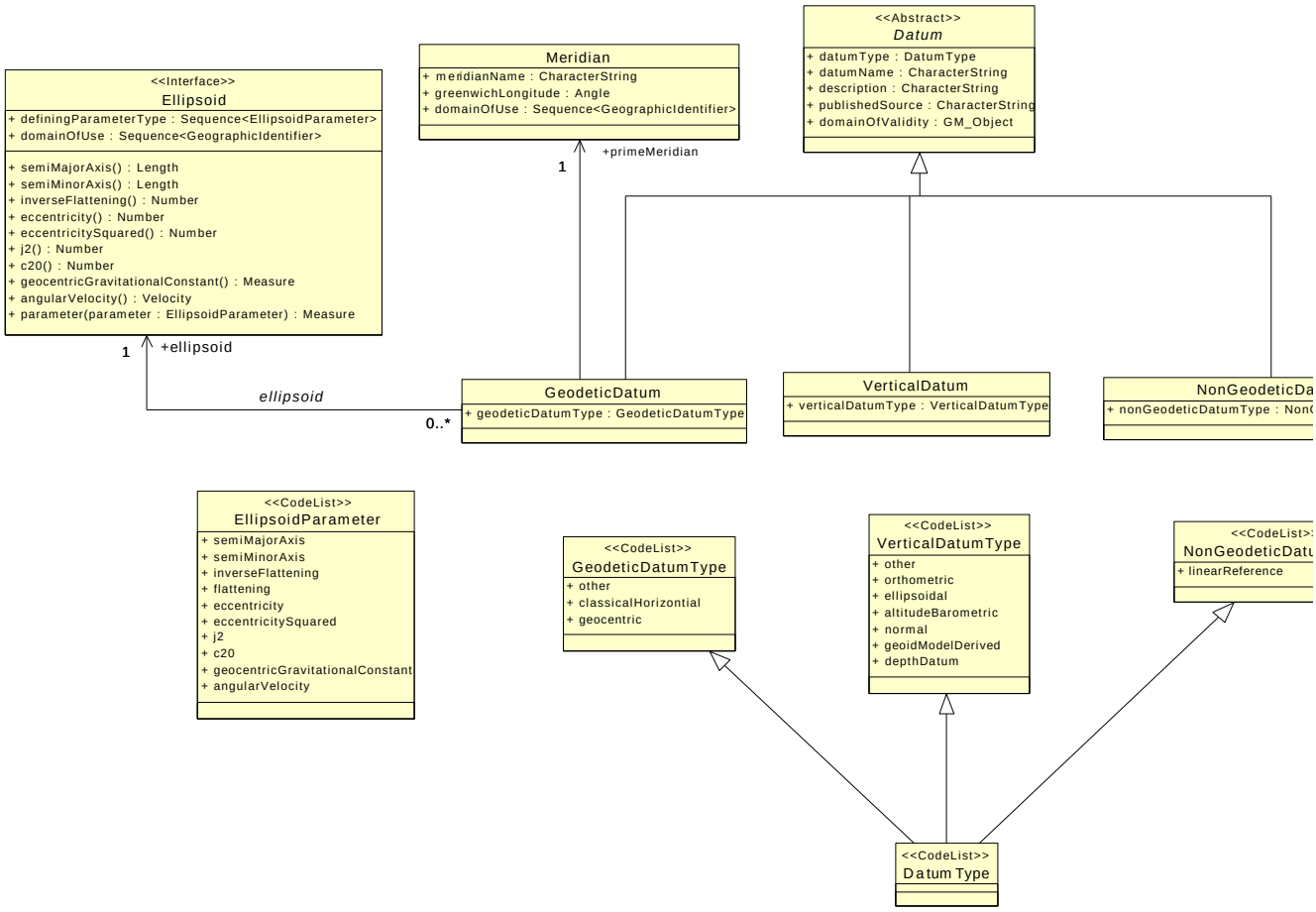

Figure 3-1. Main Class Diagram for Datum Package

3.3.1. Class name: Datum

Package the Class belongs to: Datum

Documentation: In general, a quantity or set of quantities that serve as a reference or basis for the calculation of other quantities. For the OGC abstract model, it can be defined as a set of real points on the earth that have coordinates.

A datum can be thought of as a set of parameters defining completely the origin and orientation of a coordinate system with respect to the earth (source: Spanish comment to ISO 15046-11 CD).

A textual description and/or a set of parameters describing the relationship of a coordinate system to some predefined physical locations (such as center of mass) and physical directions (such as axis of spin). The definition of the datum may also include the temporal behavior (such as the rate of change of the orientation of the coordinate axes or the rate of change of stations used in the adjustments).

Stereotype: Abstract

Hierarchy:

Superclasses: none

Associations:

<no rolename> : AbstractPoint in association comprises

<no rolename> : CoordinateReferenceSystem in association: <unnamed>

Public Interfaces:

Attributes: datumType

datumName

publishedSource

description

domainOfValidity

Attributes:

datumType : DatumType

Any one of the types of datums currently proposed as being supported by this object model. The are listed in the various code lists of this package.

datumName : CharacterString

The name of the datum being used.

publishedSource : CharacterString

This attribute is the name and version of a recognized source listing the datum identifier.

description : CharacterString

A description of the datum. It should include the following types of information concerning the datum: 1) orientation of the datum, 2) the methods used to generate the datum and 3) origin of datum.

domainOfValidity : GM_Object

The domainOfValidity attribute is a "GM_Object" that defines the geographical region where it is appropriate to use the datum.

The GM_Object is either in the corresponding geodetic coordinate system in question, or some other reference system such as WGS84 Reference Frame.

3.3.2. Class name: GeodeticDatum

Package the Class belongs to: Datum

Documentation:

The GeodeticDatum Class is an ellipsoid and a set of one or more reference points on the earth's surface. The physical location of these points is consistent with their locations on their ellipsoidal coordinate system.

A set of parameters defines the size and shape of the chosen reference ellipsoid, and its position and orientation with respect to the Earth.

NOTE: in this standard the complete definition of (any realization of) an ITRF geodetic datum must include specification of GRS80 ellipsoid.

GRS80 - An equipotential ellipsoid adopted by the International Union of Geodesy and Geophysics in 1979. The GRS80 ellipsoid is defined by semi-major axis, gravitational constant of the Earth (including the atmosphere), dynamical form factor of the Earth, and angular rotational velocity of the Earth.

ITRF - In the geodetic terminology, a reference frame (more traditionally: geodetic datum) is a set of points with their coordinates (in the broad sense) which realize an ideal reference system. The International Terrestrial Reference Frames (ITRF), consisting of the estimates of the coordinates and velocities of a set of stations, are produced by the International Earth Rotation Service (IERS) as realizations of International Terrestrial Reference System (ITRS). The parameters of GRS80 ellipsoid are to be used when expressing ITRF coordinates in the ellipsoidal coordinate system.

Hierarchy:

Superclasses: Datum

Associations:

theEllipsoid : Ellipsoid in association: ellipsoid

primeMeridian : Meridian in association: <unnamed>

Public Interfaces:

Attributes: geodeticDatumType

Attributes:

geodeticDatumType : geodeticDatumType

This attribute can be used to list the type of the geodetic datums currently proposed for this abstract model.

3.3.3. Class name: VerticalDatum

Package the Class belongs to: Datum

Documentation:

A textual description and/or a set of parameters identifying a particular reference level surface used as a zero-height surface, including its position and orientation with respect to the Earth, for any of the height types recognized by this standard.

NOTE: The examples of the height types recognized by this standard include, but may not be limited to, the following: ellipsoidal, orthometric, geoid-model-derived, altitude-barometric, depth and other.

Hierarchy:

Superclasses: Datum

Public Interfaces:

Attributes: verticalDatumType

Attributes:

verticalDatumType : verticalDatumType

This attribute lists the type of the vertical datums currently proposed for this abstract model.

3.3.4. Class name: NonGeodeticDatum

Package the Class belongs to: Datum

Documentation:

This class represents any datum that does not use the concept of reference ellipsoid, or the concept of reference gravity related surface, with the defined position and orientation within the Earth. When a nongeodetic datum supports coordinates in the "vertical" direction (along a Z-coordinate line), then the implied "non-geodetic level " (defined by the surface of zero-Zs) is conceptually different from a reference level surface that supports any of the height types recognized in this specification

Hierarchy:

Superclasses: Datum

Public Interfaces:

Attributes: nonGeodeticDatumType

Attributes:

nonGeodeticDatumType : nonGeodeticDatumType

This attribute can be used to list the type of the non-geodetic datums currently proposed for this abstract model.

3.3.5. Class name: Ellipsoid

Package the Class belongs to: Datum

Documentation:

The figure formed by the rotation of an ellipse about an axis. In this context, the axis of rotation is always the minor axis. It is named geodetic ellipsoid if the parameters are derived by the measurement of the shape and the size of the Earth to approximate the geoid as close as possible.

Stereotype: Interface

Hierarchy:

Superclasses: none

Associations:

<no rolename> : GeodeticDatum in association ellipsoid

Public Interfaces:

Attributes: definingParameterType

domainOfUse

Operations: semiMajorAxis

semiMinorAxis

inverseFlattening

eccentricity

eccentricitySquared

j2

c20

geocentricGravitationalConstant

angularVelocity

Attributes:

definingParameterType : Sequence<EllipsoidParameter>

This attribute is a list of the ellipsoid defining parameters that must each be selected from an optional list defined in the code list class; “EllipsoidParameter".

domainOfUse : Sequence<GeographicalIdentifier>

This attribute is used to specify where a particular ellipsoid should be used.

Operation: **semiMajorAxis() : Length**

Public member of: Ellipsoid

Return Class: Length

Documentation:

A query operation being used to expose the value of a given ellipsoid parameter. The operation name is the same as the parameter being queried for.

Operation: **semiMinorAxis() : Length**

Public member of: Ellipsoid

Return Class: Length

Documentation:

A query operation being used to expose the value of a given ellipsoid parameter. The operation name is the same as the parameter being queried for.

Operation: **inverseFlattening() : Number**

Public member of: Ellipsoid

Return Class: Number

Documentation:

A query operation being used to expose the value of a given ellipsoid parameter. The operation name is the same as the parameter being queried for.

Operation: **eccentricity() : Number**

Public member of: Ellipsoid

Return Class: Number

Documentation:

A query operation being used to expose the value of a given ellipsoid parameter. The operation name is the same as the parameter being queried for.

Operation: **eccentricitySquared() : Number**

Public member of: Ellipsoid

Return Class: Number

Documentation:

This operation queries for or calculates the eccentricity squared (e*2) parameter of the ellipsoid.

Operation: **j2() : Number**

Public member of: Ellipsoid

Return Class: Number

Documentation:

This operation queries a source for the "j2 parameter" of the ellipsoid.

Operation: **c20() : Number**

Public member of: Ellipsoid

Return Class: Number

Documentation:

This operation queries a source for the "c20" of the ellipsoid.

Operation: **geocentricGravitationalConstant() : Measure**

Public member of: Ellipsoid

Return Class: Number

Documentation:

This operation queries a source for the "GeocentricGravitationalConstant" of the Ellipsoid

Operation: **angularVelocity() : Velocity**

Public member of: Ellipsoid

Return Class: Velocity

Documentation:

This operation queries for the Angular Velocity parameter of the ellipsoid.

Operation: **parameter(parameter : EllipsoidParameter) : Measure**

Public member of: Ellipsoid

Return Class: Measure

Argument: EllipsoidParameter parameter

Documentation:

A general interface that can return any named parameter of the ellipsoid, whether it is stored or calculated.

The fundamental purpose of this interface is to allow the list of ellipsoid parameters to be expanded without having to expand the Ellipsoid interface.

3.3.6. Class name: ***EllipsoidParameter***

Package the Class belongs to: Datum

Documentation:

This class is a code list of the possible second defining parameters for the ellipsoid.

Stereotype: CodeList

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: semiMajorAxis

semiMinorAxis

flattening

inverseFlattening

eccentricity

eccentricitySquared

j2

c20

geocentricGravitationalConstant

angularVelocity

Attributes:

semiMajorAxis

One of the parameters that defines an ellipsoid. It is the equatorial radius of an ellipsoid.

semiMinorAxis

The polar radius of an ellipsoid.

flattening

A value used to indicate how closely an ellipsoid approaches a spherical shape. It is the ratio of the difference between the semi-major and semi-minor axis of an ellipsoid to the

semi-major axis of an ellipsoid (i.e., f=[a-b/a]). Where the terms in the previous equation are:

f - flattening

a - semi-major axis value

b - semi-minor axis value

inverseFlattening

The inverse term of the flattening term of an ellipsoid (i.e., 1/f or a/[a-b]).

eccentricity

The ratio of the distance between the center and a focus of the ellipse to the length of its semimajor axis.

The eccentricity can alternately be computed from the equation from: e = SQRT(2f-f^2)

(This definition is from the "DoD Glossary of Mapping Charting and Geodetic Terms")

eccentricitySquared

The square of the eccentricity.

j2

The j2 coefficient is one of the defining parameters of the mean earth ellipsoid and can be

related to the principal moments of inertia of the earth and c20. The mean earth ellipsoid in many respects can be considered to provide the best representation of the earth by an ellipsoid. (The definition is from the text, "Physical Geodesy" by Weikko Heiskanen and Helmut Moritz).

c20

The c20 coefficient is the largest one in the spherical harmonics expansion used to

represent the disturbing potential of the Earth's gravitational field. This coefficient represents the effect of the flattening of the Earth on the gravitational field.

(Definition is from the text, "GPS Satellite Surveying" by Alfred Leick).

geocentricGravitationalConstant

One of the defining parameters of the "satellite-age", modern ellipsoid, such as GRS80 or WGS84. It is always assumed to be in the units of [m^3/s^2].

angularVelocity

One of the defining parameters of the "satellite-age", modern ellipsoid, such as GRS80 orWGS84. It is always assumed to be in the units of [rad/s].

3.3.7. Class name: Meridian

Package the Class belongs to: Datum

Documentation:

A meridian can be thought of as the intersection of an ellipsoid by a plane containing the semi-minor axis of the ellipsoid. It is often used for the pole-to-pole arc rather than the complete closed figure.

Hierarchy:

Superclasses: none

Associations:

<primeMeridian> : GeodeticDatum in association: <unnamed>

Public Interfaces:

Attributes: meridianName

greenwichLongitude

domainOfUse

Attributes:

meridianName : CharacterString

The name of the meridian if one is known.

greenwichLongitude : Angle

The longitude of the prime meridian in the Greenwich System. The Greenwich Meridian is the meridian passing through the Royal Observatory in Greenwich, UK.

domainOfUse : Sequence<GeographicalIdentifier>

This attribute is used to specify where a particular meridian is appropriate to use.

3.3.8. Class name: GeodeticDatumType

Package the Class belongs to: Datum

Documentation:

A code list of the possible geodetic datum types that are proposed as being supported by this model.

Stereotype: CodeList

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: other

classicalHorizontial

geocentric

other

Added for extensibility of the model.

classicalHorizontial

These datums, such as ED50, NAD27 and NAD83, have been designed to support horizontal positions on the ellipsoid as opposed to positions in 3-D space. These datums were designed mainly to support a horizontal component of a position in a domain of limited extent, such as a country, a region or a continent.

geocentric

A geocentric datum is a "satellite age" modern geodetic datum mainly of global extent, such as WGS84 (used in GPS), PZ90 (used in GLONASS) and ITRF. These datums were designed to support both a horizontal component of position and a vertical component of position (through

ellipsoidal heights). The regional realizations of ITRF, such as ETRF, are also included in this category.

3.3.9. Class name: VerticalDatumType

Package the Class belongs to: Datum

Documentation:

A code list of the possible vertical datum types that are proposed for this model. The list is not intended to be exhaustive and it is left to the implementors to determine how this list should be extended.

Stereotype: CodeList

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: other

orthometric

ellipsoidal

geoidModelDerived

altitudeBarometric

depthDatum

Attributes:

other

This attribute has been included so that the model can be extended.

orthometric

A vertical datum for orthometric heights that are measured along the plumb line.

ellipsoidal

A vertical datum for ellipsoidal heights that are measured along the normal to the ellipsoid used in the definition of geodetic datum.

geoidModelDerived

A vertical datum of geoid model derived heights, also called GPS-derived heights. These heights are approximations of orthometric heights (H), constructed from the ellipsoidal heights (h) by the use of the given geoid undulation model (N) through the equation: H=h-N

altitudeBarometric

The vertical datum of altitudes or heights in the atmosphere. These are approximations of orthometric heights obtained with the help of a barometer or a barometric altimeter. These values are usually expressed in one of the following units: meters, feet, millibars (used to measure pressure levels), or theta value (units used to measure geopotential height).

depthDatum

The set of datums generated for hydrographic engineering projects where depth measurements below sea level are needed. It is often called a hydrographic or a marine datum. Depths are measured in the direction perpendicular (approximately) to the actual equipotential surfaces of the earth's gravity field, using such procedures as echo-sounding.

3.3.10. Class name: NonGeodeticDatumType

Package the Class belongs to: Datum

Documentation:

A code list of the possible non-geodetic datum types that are proposed for this model. The list is not intended to be exhaustive and it is left to the implementors to determine how this list should be extended.

Stereotype: CodeList

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: linearReference

linearReference

An example of this type of datum would be a local datum such as the mile markers along a state highway that are not referenced to a CRS.

3.3.11. Class name: DatumType

Package the Class belongs to: Datum

Documentation:

This class has been created to generate one code list of all the different datum types currently listed in the Coordinate Transformation model.

Stereotype: CodeList

Hierarchy:

Superclasses: GeodeticDatumType, VerticalDatumType, NonGeodeticDatumType

3.4. Coordinate Transform Package

This Package includes the various Coordinate Transformation (CT) classes. Algorithms for the transformations are found in the Math Transform Class. The transformations are often Geographic CT. These may be Concatenated Transformations, which are sequences of Transformation Steps. The TransformationStep class can be either Conversions or Transformations. Note, the use of a Transformation implies a change in the datum, and a Conversion implies a change to a coordinate reference system using the same datum.

In an effort not to make the following class diagram too complex, the subclasses of different types of Datum Transformations and Coordinate Conversions have been placed on separate diagrams which follows this section of the document.

The use of realization here is similar to but not equal to inheritance. An Interface does not constitute a complete class definition, but defines some basic class-like behavior, such as a set of attributes, association and / or operation signatures that form a logically consistent group. In Figure 3-2, realization is used to logically group signatures into three functionally related groups:

1. transformations that transform a point at a time (i.e., a CoordinatePoint)

2. transformations that transform a list of points at a time (no densification is assumed)

3. transformations that modify the accuracy of transformed points (i.e., modify ht accuracy of the set of DirectPositions)

In any implementation of this model, these 3 groups shall be handled as units, either implementing all of the operations in a group or none of them.

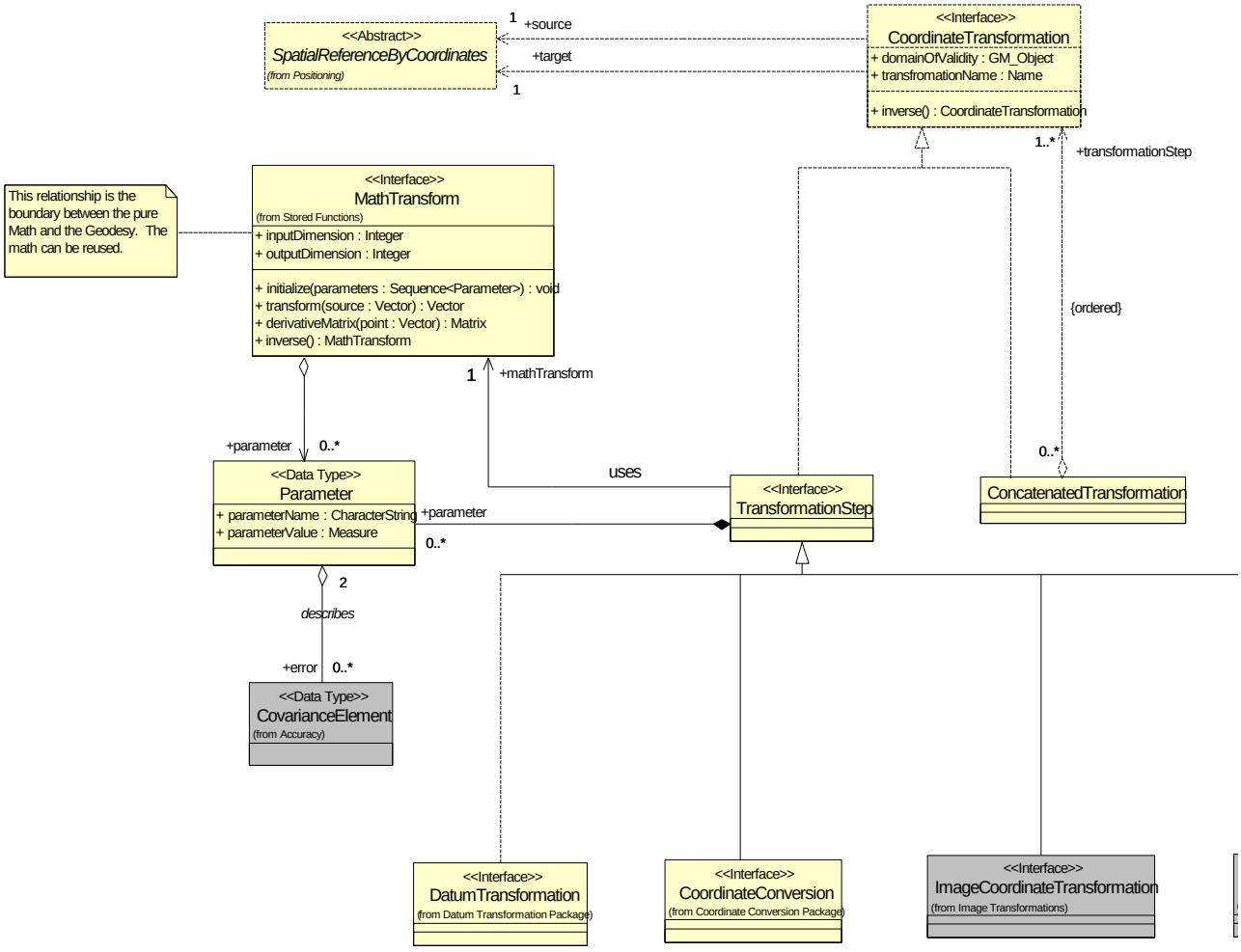

Figure 3-1. Major Class Diagram for The Coordinate Transform Package

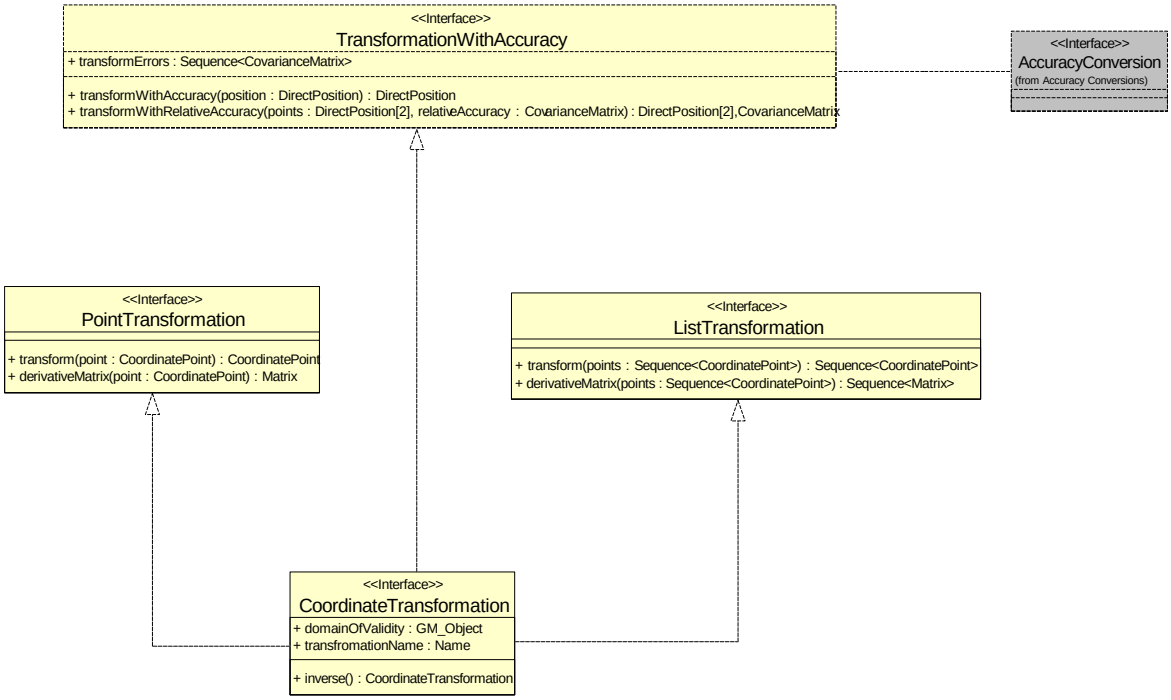

Figure 3-2. The Set of CoordinateTransformation Interfaces of the Object Model.

3.4.1. Class name: Coordinate Tranformation

Package the Class belongs to: Coordinate Transform

Documentation:

This service interface transforms a coordinate point position between two different coordinate reference systems. A coordinate transformation interface establishes an association between a source and a target coordinate reference system, and provides operations for transforming coordinates in the source coordinate reference system to coordinates in the target coordinate reference system. These coordinate systems can be ground or image coordinates. In general mathematics, "transformation" is the general term for mappings between coordinate systems (see tensor analysis).

For a ground coordinate point, if the transformation depends only on mathematically derived parameters (as in a cartographic projection), then this is an ISO conversion. If the transformation depends on empirically derived parameters (as in datum transformations), then this is an ISO transformation.

Stereotype: Interface

Hierarchy:

Superclasses: none

Associations:

<no rolename> : SpatialReferenceByCoordinates in association: source

<no rolename> : SpatialReferenceByCoordinates in association: target

<no rolename> : ConcatenatedTransformation in association: <unnamed>

Public Interfaces:

Attributes: transformationName

domainOfValidity

Operations: inverse

Attributes:

transformationName : Name

This attribute is the identifier of the coordinate transformation being used.

domainOfValidity : GM_Object

The domainOfValidity attribute is a geometry in a coordinate reference system that supports the GM_Object Interface. The CRS of this object may be the in the source, target or some standard coordinate system.

This geometry defines the geographical (or image) region where the CoordinateTransformation interface is appropriate to use.

The GM_Object interface as described in the OGC Abstract Spec (Topic Vol. 1 - Feature Geometry), includes operations that will check whether or not a given geometry or a given point is spatially within the domain of this GM_Object.

Operation: inverse() : CoordinateTransformation

Public member of: CoordinateTransformation

Return Class: CoordinateTransformation

Documentation:

The operation "inverse" returns a CoordinateTransformation that reverses the actions of this CoordinateTransformation.

The target of the inverse transform is the source of the original.

The source of the inverse transform is the target of the original.

Using the original transform followed by the inverse's transform will result in an identity map on the source coordinate space, when allowances for error are made.

3.4.2. Class name: TransformationStep

Package the Class belongs to: Coordinate Transform

Documentation:

This class transforms a coordinate point or a set of coordinate points between a source coordinate reference system and a target coordinate reference system in one step. This transformation or conversion is often used as one step in a more complex transformation used to transform a set of coordinate points.

A coordinate transformation step may be instantiated individually or may be one step in a sequence of steps linked in a concatenated ConcatenatedTransfomation Operation.

Stereotype: Interface

Hierarchy:

Superclasses: CoordinateTransformation

Associations:

mathTransform : MathTransform in association: uses

parameters : Parameter in association: <unnamed>

3.4.3. Class name: ConcatenatedTransformation

Package the Class belongs to: Coordinate Transform

Documentation:

Transforms a coordinate point or a set of coordinate points between a source coordinate reference system and a target coordinate reference system using a sequence of two or more transformation steps.

For converting between image and ground coordinates, a concatenated transformation combines zero or more Image Coordinate Transformations, exactly one Image Geometry Transformation, and zero or more ground coordinate transformations.

Hierarchy:

Superclasses: CoordinateTransformation

Associations:

<transformationStep> : CoordinateTransformation in association:<unnamed>

A ConcatenatedTransformation is a sequence of transformationSteps, each being either a single step, or another ConcatenatedTransformation. Since 1-long sequences do not cause any logical inconsistency at the behavioral level, this association was not restricted to "2 or more steps," even though that would be the naïve conclusion from the class name. The current multiplicity should also make editing of the sequence easier, since during an edit the list may well drop to 1-long. Viable implementation could allow the list to drop to 0-long with the assumption that that would constitute the identity transformation.

3.4.4. Class name: MathTransform

Package the Class belongs to: Stored Functions

Note: This package contains classes for the implementation of Stored Functions.

A Stored Function is an object that uses a finite set of data points or values and a interpolation scheme to implement a function that potentially has an infinite domain.

Documentation:

This service interface transforms a set of coordinate points to another set of coordinate points using a mathematical or stored function. A MathTransform can support either a transformation, and/or a conversion (without error) operations. The same MathTransform can be used by multiple TransformationStep objects, using different sets of parameter values for each.

Stereotype: Interface

Hierarchy:

Superclasses: none

Associations:

<no rolename> : TransformationStep in association: uses

parameter : Parameter in association <unnamed>

Public Interfaces:

Attributes: inputDimension : Integer

outputDimension : Integer

Operations: inverse

transform

derivativeMatrix

inverse

Operation: **initialize(parameters : Sequence<Parameter>) : void**

Public member of: MathTransform

Return Class: Void

Arguments: sequence<Parameter> parameters

Documentation:

Initializes the Math Transformation object so it can be used correctly.

The initialize function is often called from the object constructor to fill in the appropriate attribute values based on the constructor's parameters.

A MathTransform that is not initialized will use default values for the parameters. During initialize, the MathTransform object will change the values of all parameters passed in and all parameters dependent on the passed parameters. Unreferenced parameters will retain their previous values (which may or may not be the default if this object has been previously initialized).

For example, if the transform is an Affine Transformation, and the parameters sent are the various rotations of a unitary matrix, then the initialize operation will calculate the elements of the Affine Transformation.

One use of the initialize operation is to reuse the math transform object without reconstruction, such as changing a single parameter to obtain the inverse MathTransform. This operation may be used in conjunction with a copy constructor when the original MathTransform object is needed in its current state.

Operation: **transform(source : Vector) : Vector**

Public member of: MathTransform

Return Class: Vector

Arguments: Vector source

Documentation:

This operation transforms a coordinate value for a point given in the source coordinate system to coordinate value for the same point in the target coordinate system.

In an ISO conversion, the transformation is accurate to within the limitations of the computer making the calculations.

In an ISO transformation, where some of the operational parameters are derived from observations, the transformation is accurate to within the limitations of those observations.

Operation: **derivativeMatrix(point : Vector) : Matrix**

Public member of: MathTransform

Return Class: Matrix

Arguments: Vector point

Documentation:

The derivativeMatrix operation determines the matrix of partial derivatives of the transformation result coordinate points with respect to the input coordinate points at a specified position. This matrix is the multi-dimensional equivalent of a tangent.

Operation: **inverse() : MathTransform**

Public member of: MathTransform

Return Class: Vector

Arguments: Vector source

Documentation:

The operation "inverse" returns a new MathTransform object that is already initialized with the appropriate parameters so that its transform method reverses the transform method of this (the original) MathTransform.

For example, if the original MathTransform object was a Matrix multiply, the inverse transformation is multiplication by of the original Matrix.

3.4.5. Class name: Parameter

Package the Class belongs to: Coordinate Transform

Documentation:

This class contains numerical values that can be used by a transformation or conversion as control parameters (such as polynomial coefficients, or physical measurements). These parameters are generally used in the construction or initialization of the mathematical transforms used by the TransformationStep.

Hierarchy:

Superclasses: none

Stereotype: Data Type

Associations:

<no rolename> : TransformationStep in association: <unnamed>

<no rolename> MathTransform in association: <unnamed>

<error> : CovarianceElement in association: <describes>

Note: A single Covariance Element object can define the variance in the value of the associated Parameter. A set of Covariance Element objects can define a Covariance Matrix across a set of parameters whose values have correlated errors. Such a Covariance Matrix can be across multiple parameters, but not necessarily all the parameters used by one Transformation Step subclass or object. Such a Covariance Matrix can also be across multiple parameters used by multiple Transformation Step objects of the Image Geometry Transformation subclass. The error in the value of a Parameter object is assumed to be zero when there is no Covariance Element object associated with that Parameter object.

Public Interfaces:

Attributes: parameterName

parameterValue

Attributes:

parameterName : CharacterString

The parameterName is a common name given to a parameter used in a coordinate transformation. Examples of names commonly given to parameters used in coordinate conversions include: False Northing, False Easting, Latitude of the Origin, Longitude of the Origin, and Scale Factor. Examples of parameter names for Datum Transformations include: X Axis Translation, Y Axis Rotation, and Scale Differences.

parameterValue : Measure

The numerical values of this parameter used are numerical quantities that are required in order to describe completely the characteristics of a CRS or the relationship between two CRSs, plus a specification of the physical units when applicable.

3.4.6. Class name: ImageCoordinateTransformation

Package the Class belongs to: ImageTransformations

Documentation:

This class transforms image position coordinates between two different image coordinate reference systems, usually based on the same original image. Such transformations are required when different images are derived from an original image, such as by image cutting, magnification, rotation, and/or rectification. This class is applicable to 2-D images (not to 3-D images). Subtypes of this class include polynomial coordinate transformations and image rectification transformations.

Stereotype: Interface

Hierarchy:

Superclasses: TransformationStep

3.4.7. Class name: ImageGeometryTransformation

Package the Class belongs to: ImageTransformations

Documentation:

This class transforms position coordinates between an image coordinate system and a three-dimensional ground coordinate reference system. The image coordinate system is usually two-dimensional, but might be three-dimensional in the future. Three dimensional ground coordinates can be transformed to image coordinates (either 2-D or 3-D). (However, 2-D ground coordinates cannot be transformed to either 2-D to 3-D image coordinates.) Two-dimensional image coordinates can be transformed to ground coordinates (2-D to 3-D) only when additional information is supplied. This additional information can take the form of elevation data, ground shape (and position) data, or stereoscopic image data.

Stereotype: Interface

Hierarchy:

Superclasses: TransformationStep

3.4.8. Class name: PointTransformation

Package the Class belongs to: Coordinate Transform

Documentation:

This interface defines coordinate transformation operations that transform single a coordinate between two different coordinate reference systems. The return parameter is a transformed coordinate.

Stereotype: Interface

Hierarchy:

Superclasses: none

Public Interfaces:

Operations: transform

derivativeMatrix

Operation: **transform(point : CoordinatePoint) : CoordinatePoint**

Public member of: PointTransformation

Return Class: CoordinatePoint

Arguments: CoordinatePoint point

Documentation:

This operation transforms a coordinate value for a point given in the source coordinate system to coordinate value for the same point in the target coordinate system.

In an ISO conversion, the transformation is accurate to within the limitations of the computer making the calculations.

In an ISO transformation, where some of the operational parameters are derived from observations, the transformation is accurate to within the limitations of those observations.

Operation: **derivativeMatrix(point : CoordinatePoint) : Matrix**

Public member of: PointTransformation

Return Class: Matrix

Arguments: CoordinatePoint point

Documentation:

This operation determines the matrix of partial derivatives of the transformation result coordinates with respect to the input coordinates, at the specified position in the source coordinate reference system. This matrix is the multi-dimensional equivalent of a derivative or a tangent.

3.4.9. Class name: ListTransformation

Package the Class belongs to: Coordinate Transform

Documentation:

This interface defines coordinate transformation operations that transform a list of coordinates for each call, between two different coordinate reference systems. The return is a list of new transformed coordinates in the same order and number as the input list.

Stereotype: Interface

Hierarchy:

Superclasses: none

Public Interfaces:

Operations: transform

derivativeMatrix

OperationName: transform(points : Sequence<CoordinatePoint>) : Sequence <CoordinatePoint>

Public member of: ListTransformation

Return Class: Sequence<CoordinatePoint>

Arguments: Sequence<CoordinatePoint> points

Documentation:

This operation transforms coordinate values for a sequence of DirectPositions given in the source coordinate system to coordinate values for the same sequence of DirectPositions in the target coordinate system. The returned list contains the same number of points in the same order as the input list (no densification is performed).

OperationName : derivativeMatrix(points : Sequence<CoordinatePoint>) : Sequence <Matrix>

Public member of: ListTransformation

Return Class: Sequence<Matrix>

Arguments: Sequence<CoordinatePoint> points

Documentation:

This operation derives the matrices of the partial derivatives of the transformation result coordinates with respect to the input coordinates. Each matrix is the multi-dimensional equivalent of a derivative or a tangent. This operation is similar to the corresponding operation in the PointTransformation Interface, but works on a specified sequence of DirectPositions in the source coordinate reference system.

3.4.10. Class name: TransformationWithAccuracy

Package the Class belongs to: Coordinate Transform

Documentation:

This interface defines operations that compute error estimates for transformed point position coordinates. These operations also transform the point position coordinates. The input and output positions are of type DirectPosition, since this type includes position error data. However, each DirectPosition is associated with a SpatialReferenceByCoordinates, which is not needed with the input and output positions.

The output accuracy data always combines the error contributions from all known error sources, including:

- Position errors in the input point coordinates, input in "Direct Position" objects

- Errors in the transformation parameters, recorded in "Covariance Element" objects associated with objects of the "Parameter" class

- Errors in the transformation computations, recorded in the transformErorrs attribute of this class.

Stereotype: Interface

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: transformErrors

Operations: transformWithAccuracy

transformWithRelativeAccuracy

Attributes:

transformErrors : Sequence<CovarianceMatrix>

This attribute contains estimates of the computation errors committed in performing the various "transform" operations, by the specific implementation of the specific subclass of the Coordinate Transformation class. These Covariance Matrices reflect the errors in various coordinate systems, including the source and target coordinate systems.

Note: In some cases, these error estimates could include the combined effects of the error estimates in some or all the parameter values within a set of "Parameter" objects. This set is one used by one or more objects of the TransformationStep class, such as for a set of images that were triangulated together. Such inclusion would replace the CovarianceElement accuracy data otherwise associated with objects of the "Parameter" class.

Operation: transformWithAccuracy(position : DirectPosition) : DirectPosition

Public member of: TransformationWithAccuracy

Return Class: DirectPosition

Arguments: DirectPosition position

Documentation:

This operation transforms the specified position of a point in the source coordinate system to the corresponding position in the target coordinate system. This operation also computes absolute position error estimates (or absolute accuracy) for the transformed position returned by this operation.

Operation: transformWithRelativeAccuracy(points : DirectPosition[2]) : DirectPosition[2],CovarianceMatrix

Public member of: TransformationWithAccuracy

Return Class: DirectPosition[2], CovarianceMatrix

Arguments: DirectPosition[2] points

CovarianceMatrix relativeAccuracy

Documentation:

This operation transforms the specified positions of exactly two points in the source coordinate system to the corresponding two positions in the target coordinate system. This operation also computes relative position error estimates (or relative accuracy) between the two transformed positions returned by this operation. The input and output Covariance Matrices specify the relative accuracies between the specified pair of points.

3.5. Datum Transformation Package

This package shows how we classify Datum Transformations. Note that in the class, "GeoidEllipsoid-Transformation", the source and target CS are different types of coordinate systems.

Also note that a Gravity surface Transformation can be represented various ways, (e.g. as a grid of values of offset from an ellipsoid or as a mathematical surface).

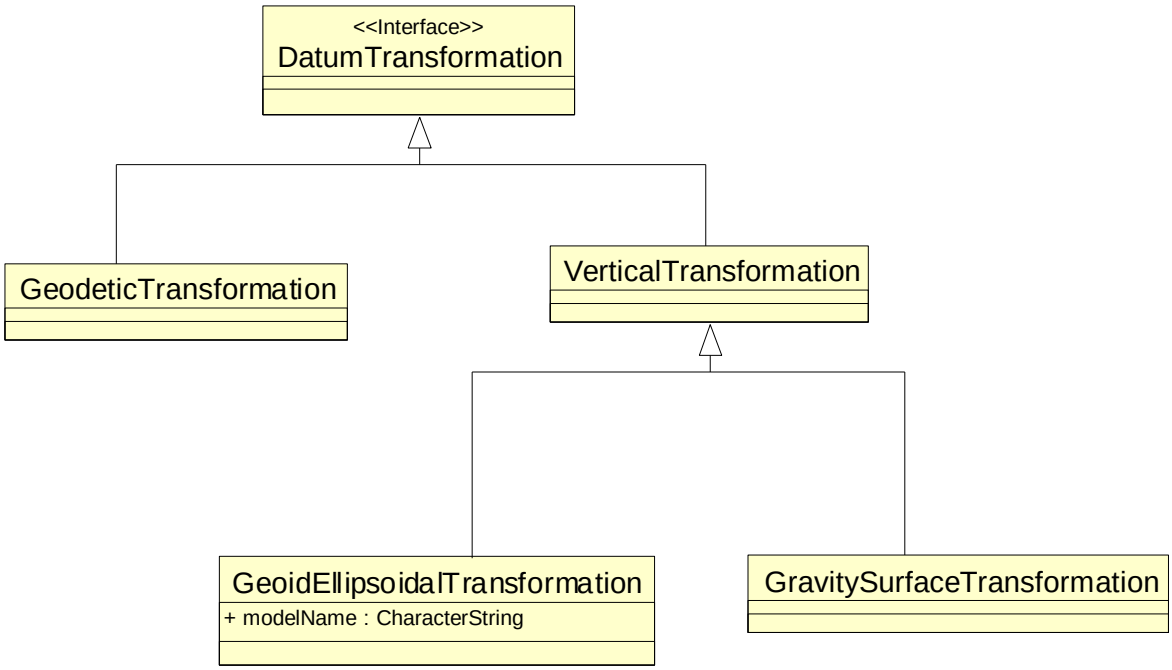

Figure 3-1. Class Diagram showing the Datum Transformation Package.

3.5.1. Class name: DatumTransformation

Package the Class belongs to: Datum Transformation

Documentation: A transformation between the coordinate points referenced to the source datum and the coordinate points referenced to the target datum. An assumption is that the difference between the source and target coordinate systems is solely the datum.

Stereotype: Interface

Hierarchy:

Superclasses: TransformationStep

3.5.2. Class name: GeodeticTransformation

Package the Class belongs to: Datum Transformation

Documentation: This class transforms coordinates between two geodetic datums.

Hierarchy:

Superclasses: DatumTransformation

3.5.3. Class name: VerticalTransformation

Package the Class belongs to: Datum Transformation

Documentation: Services that relate one elevation model to another elevation model. Horizontal coordinates are unchanged by this class of transformation.

Hierarchy:

Superclasses: DatumTransformation

3.5.4. Class name: GeoidEllipsoidalTransformation

Package the Class belongs to: Datum Transformation

Documentation: A transformation from the surface of the ellipsoid to the surface of the geoid. In this transformation, latitude and longitude are unchanged, only the elevation is changed. This is also a way of expressing a Geoid model.

The GeoidEllipsoidialTransformation provides the distance of the geoid above or below the associated reference ellipsoid.

Hierarchy:

Superclasses: VerticalTransformation

Public Interfaces:

Attributes: modelName

Attributes:

modelName : CharacterString

The name of the geoid undulation model.

3.5.5. Class name: GravitySurfaceTransformation

Package the Class Belongs to: Datum Transformation

Documentation: A mapping of elevations between two gravity-related surfaces. Some examples might include: a vertical offset that can be applied to a MSL elevation at a given latitude and longitude to convert it to the depth below the surface of a lake, a vertical offset that might be applied to a Mean Lower Low Water depth to convert it to a High Water depth."

Hierarchy:Superclasses: VerticalTransformation

3.6. Coordinate Conversion Package

This package is a specialization of the Coordinate Transform Package. It has been included to show the other major of types of transformations supported by the model. That is transformations that involve no shift in the datum when transformation a CoordinatePoint or a set of CoordinatePoints.

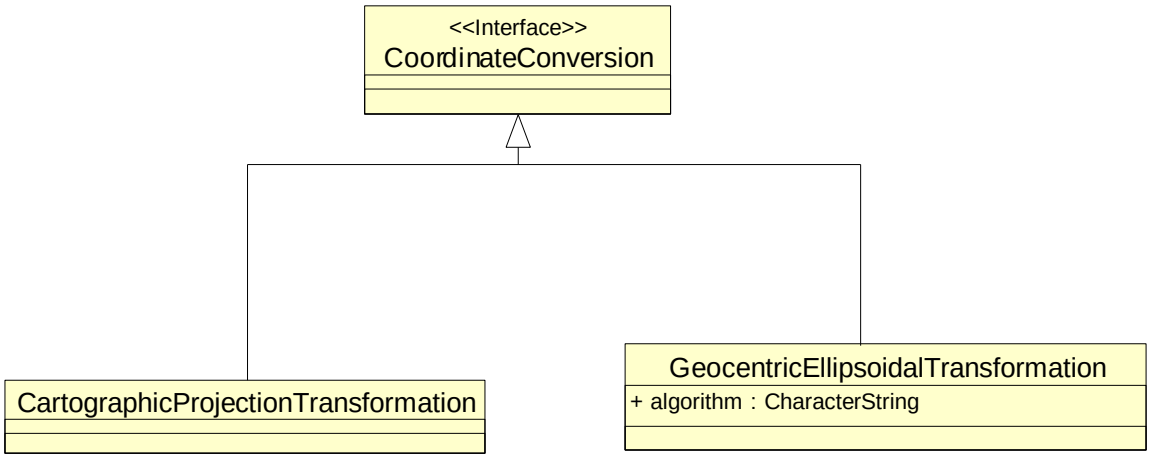

Figure 3-1. Main Class Diagram for Coordinate Conversion Package

3.6.1. Class name: CoordinateConversion

Package the Class belongs to: Coordinate Conversion

Documentation:

A transformation step where the source and target CRSs share the same datum.

Stereotype: Interface

Hierarchy:

Superclasses: TransformationStep

3.6.2. Class name: CartographicProjectionTransformations

Package the Class belongs to: Coordinate Conversion

Documentation:

This class provides transformation services between ellipsoidal and cartographic projections. Ellipsoidal height values remain unchanged.

Hierarchy:

Superclasses: CoordinateConversion

3.6.3. Class name: GeocentricEllipsoidalTransformation

Package the Class belongs to: Coordinate Conversion

Documentation:

This class provides a transformation service to convert between ellipsoidal coordinates (i.e., latitude, longitude and elevation) and geocentric Cartesian coordinates.

Hierarchy:

Superclasses: CoordinateConversion

Public Interfaces:

Attributes: algorithm

Attributes:

algorithm : CharacterString

This attribute lists the algorithm to convert between Cartesian and ellipsoidal coordinates.

3.7. References

[1] ISO 15046 Part 11 Spatial Referencing by Coordinates Committee Draft, November 10, 1998.

[2] Whiteside, Arliss. 1998 "Shared Unified Modeling Language Packages for Accuracy", OGC Change Proposal. Document Number 99-038r1c, May 1999

[3] Open GIS Consortium, 1999. Topic 1, Abstract Specification for Feature Geometry, Wayland, Massachusetts. Available via the WWW as <http://www.opengis.org/techno/specs.htm>.

4. Future Work

Areas needing work include:

1. The handling of local coordinate systems (Table 3-1) and how this effort can be tied into the handling of image coordinate systems as currently found in Topic Volume 16 (7).

2. The modeling of vertical coordinate systems and their use of vertical datums.

3. The treatment of metadata in the CT object model. This effort will need to be coordinated with the IES SIG and will need to stay abreast of the ISO/TC 211 efforts to publish an international standard on geospatial metadata.

4. Alignment with other Topic volumes including 1 (Feature Geometry), 3 (Location), 6 (Coverages) and 7 (Earth Imagery). Future work will align all these sections to a more general and abstract reference model.Topics 1, 3 and 6 deal with coordinate systems that extend the geodetic coordinate systems presented in this Topic.

5. Well Known Structures

The class diagrams that comprise the CTS object model contain many basic data types and unit of measures that have been placed in appendices A and B of this Topic Volume. Other well known structures used in this topic volume are defined in the text specification for the various classes. The accuracy classes are listed in the Image Transformation Services Topic Volume (7).

Other Well Known Structures of this Topic such as Named Datums and Projections, are to be left for the Implementation Specification. It is assumed that the OfficialName of each SRS will be sufficient to convey its entire structure.

6. Appendix A. An introduction to coordinate system geodesy

6.1.1. Model of the Earth

The earth has a complex surface. A simpler surface is produced if the earth's topography is removed. The gravity surface approximating to mean sea level (the "geoid") is taken by geodesists the shape of the earth. But even when stripped of all of its topography, the earth's internal composition gives rise to local gravity anomalies which cause the geoid to be irregular. Because of these irregularities the geoid is a difficult surface on which to describe or relate locations. Geodesists therefore employ a model of the earth using a relatively simple mathematical figure to approximate the geoid. Classical navigators assumed the earth to be spherical. It is now known that an oblate ellipsoid (or spheroid) is a better approximation of the shape of the earth. See Figure 6-1.

Model of the Earth

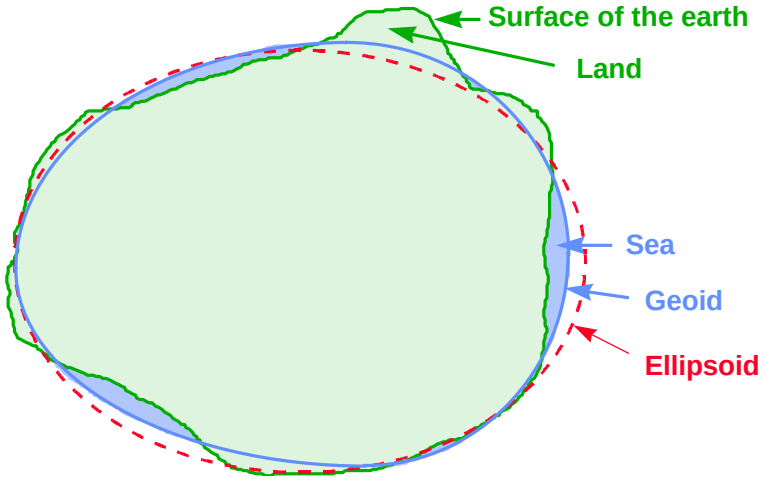

Figure 6-1. Model of the Earth

Many determinations of the best-fitting ellipsoid have been made. Several hundred ellipsoids have been defined for scientific purposes and about 30 are in use today for mapping. Examples include Clarke 1866, International 1924 and the Geodetic Reference Spheroid of 1980 (GRS80).

The size and shape of an oblate ellipsoid can be described through many parameters. Only two parameters are needed to describe both size and shape as long as at least one of these is linear to define dimension. In classical geodesy it was usual to define the ellipsoid though the semi-major axis (a) and semi-minor axis (b), or through the semi-major axis and the inverse flattening (1/f) (where f is a simple function of a and b). In modern geodesy it is practice to define the semi-major axis and several parameters describing the earth's gravitational field and rate of rotation, from which 1/f is derived.

6.1.2. Geodetic Datum

Defining an ellipsoid is in itself insufficient for coordinate values describing a point on the earth to become unique. It is also necessary to define the spatial relationship (position and orientation) between the chosen ellipsoid and the geoid. This is achieved through the definition of a "geodetic datum". In modern geodesy it is usual to define the position and orientation of the ellipsoid relative to the center of the earth. Classically, an ellipsoid was chosen and fitted to a particular area of interest, for example a country, either at a single point or as the best fit between several points on the earth's surface. Such ellipsoids did not necessarily fit in other parts of the earth. See Figure 6-1.

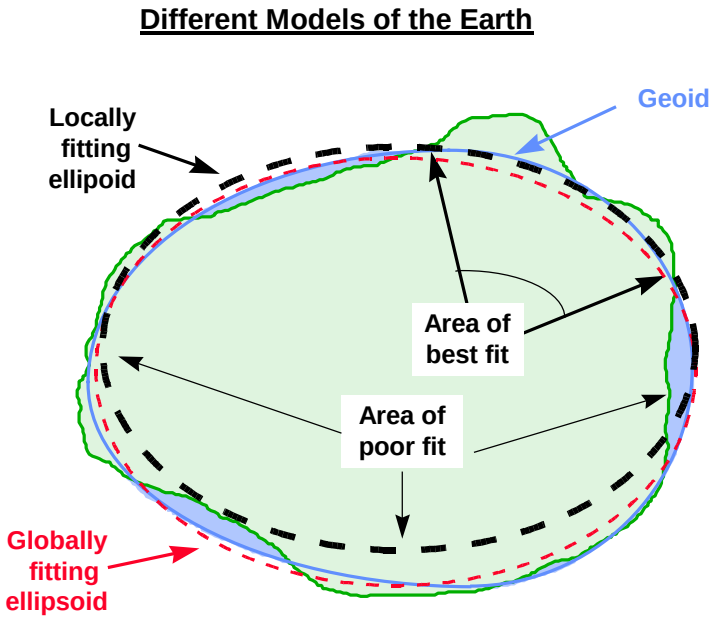

Figure 6-1. Different Models of the Earth

6.1.3. Latitude and Longitude

Ellipsoidal coordinate systems use axes of latitude and longitude. Latitude of a point is defined as the angle subtended between the ellipsoid's equatorial plane and a line from the point drawn perpendicular to the ellipsoid surface. (Strictly, this is the geodetic latitude. Other types of latitude exist. These are important in geodesy but for practical geographic information processing purposes it is generally not necessary to consider them. They are therefore not discussed further in this document). By convention, latitude is positive if north of the equator, negative if south. If the spatial relationship between the ellipsoid and the geoid is changed, or if the size or shape of an ellipsoid is changed, the latitude of a point will change. See Figure 6-1.

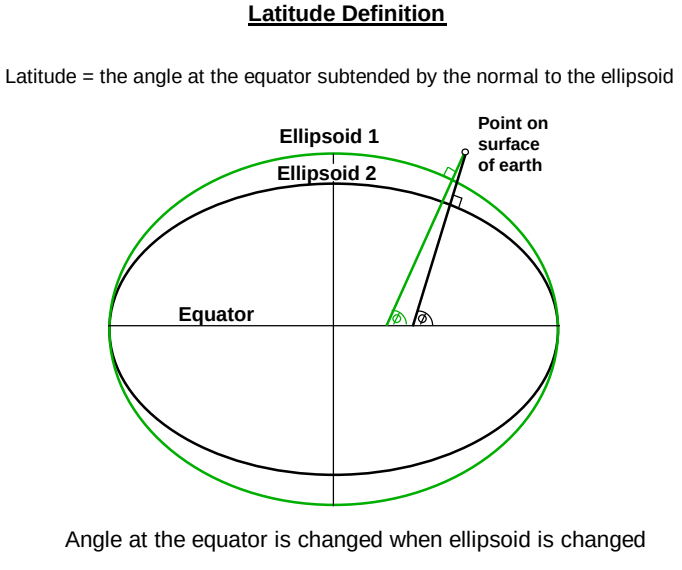

Figure 6-1. Latitude Definition

Longitude is defined to be the angle measured about the minor (polar) axis of the ellipsoid from an origin or prime meridian plane to the plane of the meridian through the point, positive if east of the prime meridian and negative if west. Unlike latitude for which there is a natural origin in the equator, there is no feature on the ellipsoid which forms a natural origin for the measurement of longitude. The zero longitude can be defined to be any meridian. Historically, nations have used the meridian through their national astronomic observatory, giving rise to several prime meridians. By a 19th century international convention the meridian through Greenwich, England, is the standard prime meridian. However prime meridians other than Greenwich are still found in use around the world. To be integrated with other geographical coordinates, the relationship between the local prime meridian and Greenwich meridian must be known.

6.1.4. Height

Latitude and longitude form a two-dimensional coordinate system on the surface of the ellipsoid. To fully define the spatial location of an object it is necessary to add a vertical ordinate. This is measured outwards from the ellipsoidal surface along a perpendicular to the surface and is known as *ellipsoidal height.* Latitude, longitude and ellipsoidal height form a three-dimensional coordinate system.

But most heights encountered in geographic information processing are not ellipsoidal. They are measured from the geoid (or an estimation of the geoid), and measured along the direction of the gravity field of the earth. These we will call *gravity-related heights*. Geodesists recognize several types of gravity related height, with names such as orthometric height and normal height, differentiated by the assumptions made about the gravity field. These issues are beyond the scope of this note. For geographic information processing the differences between various types of gravity-related height are usually insignificant and can be ignored.

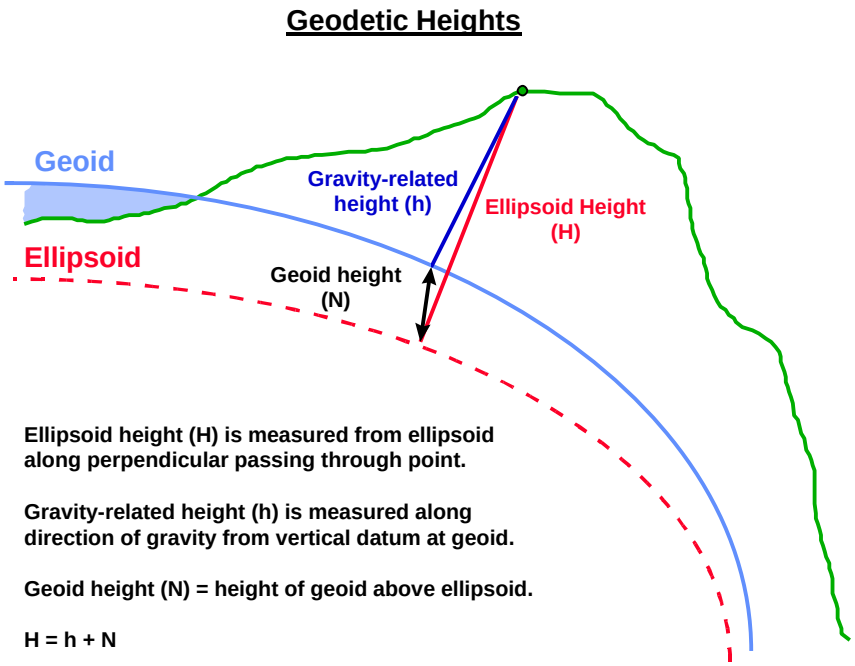

Figure 6-1. Geodetic Heights

The separation in the vertical plane between the ellipsoid and the geoid is known as the *geoid height* or "geoidal undulation". At any point there is a geoid height for every geodetic datum relevant to that point. For a particular geodetic datum, the geoid height value changes continuously with change in horizontal position. An exact value for geoid heights relative to the ellipsoidal surface for a geodetic datum is not usually known. However, for some geodetic datums a model of the geoid heights has been built and approximate geoid heights can be interpolated across the model. With a locally well-fitting and well-positioned ellipsoid, the values of geoid height will be

close to zero. For global geodetic datums such as WGS 84, geoid height values range between +50 to -50 metres. For non-global geodetic datums covering extensive areas, especially if near mountains or if using an old ellipsoid adopted from measurements elsewhere in the world, the absolute values of geoid height can exceed 200 metres in extreme cases.

Gravity-related vertical coordinate systems are measured relative to a vertical datum. The vertical datum will be taken to be the geoid, and will usually be mean sea level at a particular location or series of locations over a particular period of time. As with geodetic datums, the parameters required to define a vertical datum can vary and can be complex, but for practical geographical information processing whilst the definition of the vertical datum may be useful it is not essential. However it is critical that the vertical datum is identified. This is because if the datum surface is changed, the values of heights measured from that surface will change.

6.1.5. Geocentric Coordinates

Latitude, longitude and ellipsoidal height can be easily transformed into a three-dimensional cartesian coordinate frame where the cartesian origin is coincident with the center of the ellipsoid. Such a system is known as a **geocentric** coordinate system. Indeed in modern geodesy this is the starting point for a reference system definition, with ellipsoidal and then projected coordinates being derived from the geocentric system.

Relationship between geocentric and ellipsoidal coordinate systems

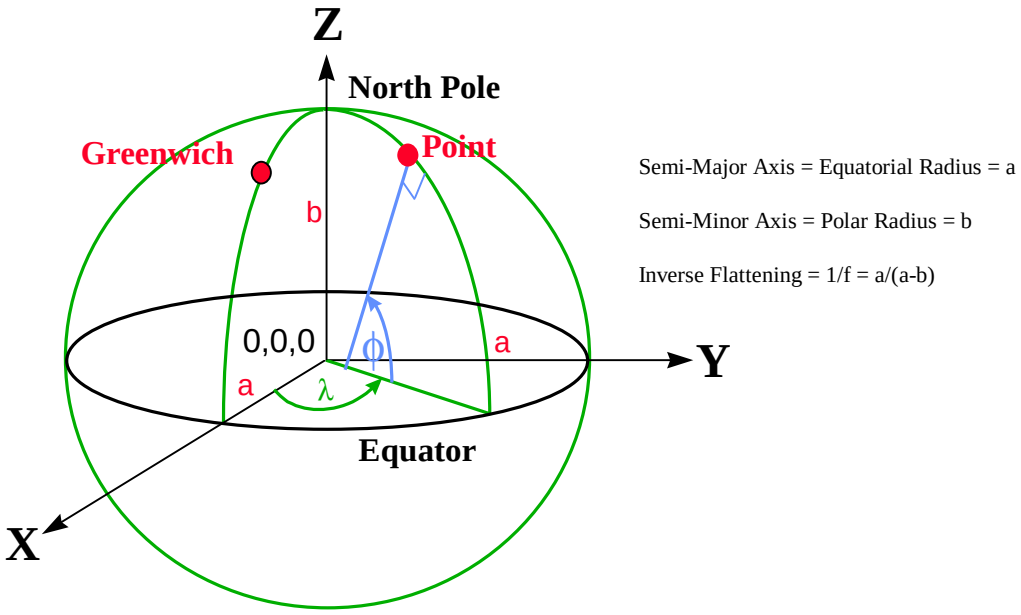

Figure 6-1. Relationship between geocentric and ellipsoidal coordinate systems

Equations for changing latitude, longitude and ellipsoidal height to and from geocentric coordinates (using the same geodetic datum) can be found in standard geodetic texts. The only parameters required are for the size and shape of the ellipsoid.

6.1.6. Geodetic Transformations

Ellipsoidal 3D and geocentric coordinates in one coordinate system can be transformed to another geodetic coordinate system if the relative positions of the two systems can be described. This is frequently achieved through a simple three-dimensional transformation between geocentric coordinate systems. For ellipsoidal 3D coordinates, the relationship between ellipsoidal and geocentric coordinates is straightforward and can be included in the transformation process.

The same procedures can be used to transform ellipsoidal 2D coordinates, but exactly only if an orthometric height is known or assumed and can first be transformed to an ellipsoidal height

through the application of the geoid height. In practice, for geographical information processing, the method can sometimes be used within the general accuracy requirements of the user without knowledge of the geoid height. Alternative methods which transform directly between ellipsoidal 2D coordinate systems are also available.

The 3-translation model assumes that the axes of the geocentric coordinate systems are parallel, and that the scales of the coordinate systems are identical. Both of these assumptions in general are not true. The 3-translation model is therefore an approximation. The approximation can be improved if a 7-parameter Helmert transformation is used. This adds three rotations and a scaling to the three translations. A difficulty with practical implementation of this is that there two opposite conventions in use for the sign of the rotations. For the transformation to be correctly implemented it is essential that the mathematical formulation is known and that parameter values are consistent with this formulation.

But even the 7-parameter transformations do not model distortions inherent in the realization of the geodetic datum. This results in different sets of transformation parameter values being available between any two coordinate systems, with the sets having applicability to different (possibly overlapping) geographical areas. Alternatively, different translations my co-exist for a particular area, each being used for specific applications. It cannot be assumed that publicly available geodetic transformation data for an area can be used for all purposes. It is the fact that geodetic transformation parameters are measured and therefore not exact that distinguishes transformations from projections or unit conversions.

Some national authorities suggest a two-step transformation between coordinate systems with the coordinates related to the source coordinate system being transformed to the target coordinate system via an intermediate coordinate system.

6.1.7. Map Projections

The lattice of geographical coordinates on the ellipsoid is not an especially convenient surface for spatial manipulation. Far simpler is a plane. A map projection converts ellipsoidal 2D coordinates (latitude/longitude) into plane or **projected** coordinates (northing/easting). The resulting projected coordinates remain geolocated. They are, however, dependent upon the source geographic coordinate system. "Northing" and "easting" are terms used to describe the direction of the axes of a projected coordinate system. The axes may be given alternative names and/or be given in a user-defined order.

The ellipsoid is not a developable surface, that is it cannot be transformed onto a plane without distortion. Mathematical cartographers have studied the distortions and produced projection types (conversion formulae) in which the behavior of the distortions is controlled. Projection methods in common use have names such as "Albers Equal Area" (which controls distortion of area), "Lambert Conformal" (which controls angular distortion), etc. Many of these methods were first developed for the sphere and then adapted for the ellipsoid. The complexities of the ellipsoidal adaptation have often led to several solutions. Some of these may differ only very slightly in their result and for practical purposes within geographic information processing be considered identical. For example, the "Gauss-Kruger" and "Gauss-Boaga" ellipsoidal forms of the "Transverse Mercator" method are for all practical purposes identical. Others, sometimes unfortunately with the same name, may produce differences which are of an unacceptable magnitude. For example the ellipsoidal formulas for the "Oblique Stereographic" method commonly used in the USA are not the same as those encountered in Europe. Errorless coordinate conversion requires that the projection method, its projection parameters, and the projection parameter values are all clearly defined and mutually compatible.

The mathematical conversion of ellipsoidal coordinates to plane requires the dimensions of the ellipsoid and values for a set of parameters specific to the projection type. Geodetic datum parameters are not a necessary part of the mathematical manipulation. But note that if the input geographical coordinates are not related to a geodetic datum then the output plane coordinates will also be ambiguous. The ambiguity will not be significant if the geographical processing requires an accuracy of no better than about 1 km (mapping at scales smaller than 1:1,000,000), but for detailed analyses properly geolocated coordinates will be required.

7. Appendix B. Unit of Measure Package

This package is a support package for the Coordinate Transformation object model. It contains classes that would probably be used as part of implementing a CT Service.

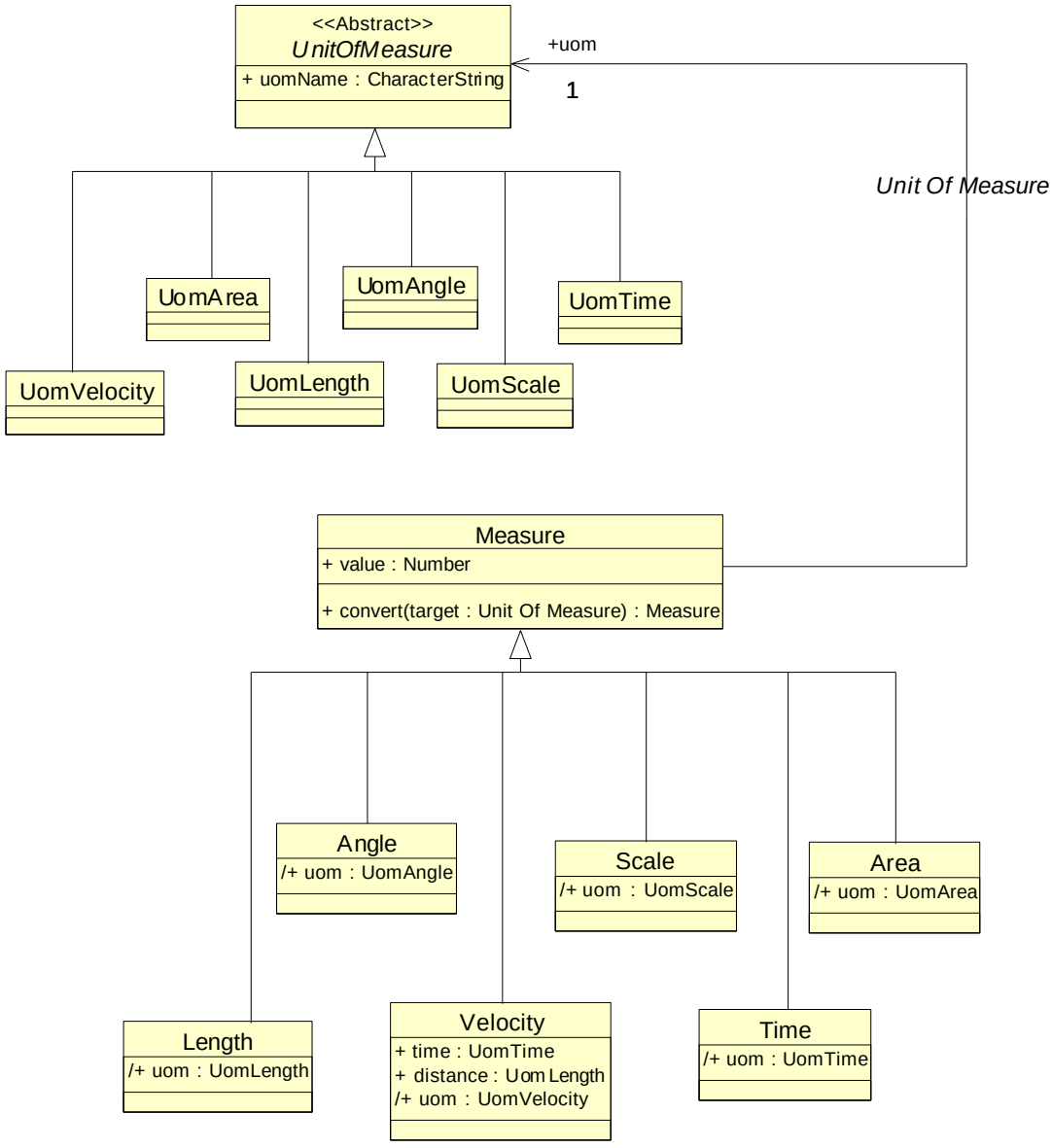

Figure 7-1. Main Class Diagram for Unit of Measure Package

7.1.1. Class name: Unit Of Measure

Package the Class belongs to: Unit of Measure

Documentation:

Any of the systems devised to measure some physical quantity such as distance or area or a system devised to measure such things as the passage of time.

Stereotype: Abstract

Hierarchy: Superclasses: none

Associations:<no rolename> : Measurement in association UOM

Public Interfaces:Attributes: uomName

UomName: CharacterString

The name(s) assigned to a particular unit of measure. Examples would include the following: 1) for uomArea - square feet, 2) for uomTime - seconds, 3) for uomArea - miles and 4) uomAngle - degrees.

7.1.2. Class name: UomScale

Package the Class belongs to: Unit of Measure

Documentation:

Any of the measuring systems commonly used to measure scale, or the ratio between quantities with the same units. Scale factors are often unitless.

Hierarchy:

Superclasses: UnitOfMeasure

7.1.3. Class name: UomLength

Package the Class belongs to: Unit of Measure

Documentation: Any of the measuring systems to measure the length, distance between two entities. Example are the English System of feet and inches or the metric system of millimeters, centimeters and meters.

Hierarchy:

Superclasses: UnitOfMeasure

7.1.4. Class name: UomAngle

Package the Class belongs to: Unit of Measure

Documentation:

The measuring system(s) commonly used to measure angles. In the U.S., the sexigesimal system of degrees, minutes and seconds is frequently used. The radian measurement system is also used to a limited degree. In other parts of the world the grad angle measuring system is used.

Hierarchy:

Superclasses: UnitOfMeasure

7.1.5. Class name: UomTime

Package the Class belongs to: Unit of Measure

Documentation:

Any of the systems or methods of measuring or reckoning the passage of time and/or date, (e.g., seconds, minutes, days, months).

Hierarchy:

Superclasses: UnitOfMeasure

7.1.6. Class name: UomArea

Package the Class belongs to: Unit of Measure

Documentation:

Any of the measuring systems commonly used to measure area. Common units include square length units, such as square meters and square feet. Other common unit include acres (in the U.S.) and hectares.

Hierarchy:

Superclasses: UnitOfMeasure

7.1.7. Class name: UomVelocity

Package the Class belongs to: Unit of Measure

Documentation:

Any of the systems or methods used to measure motion, usually measured as the change in position during a given time interval (e.g., m/sec^2).

Hierarchy:

Superclasses: UnitOfMeasure

7.1.8. Class name: Measure

Package the Class belongs to: Unit of Measure

Documentation:

The result from performing the act or process of ascertaining the extent, dimensions, or quantity of some entity.

Subclasses of Measure will have to specify which class of UnitOfMeasure that they use. That is, a length measure must use a length unit of measure, and so forth. This restriction is given in the model by a specialization of the role name "uom" which is logically a derived attribute or pseudo-attribute (see UML-RM p166). Note that by substitutability, a specialized member definition must be consistent with the member definition at any superclass (see UML-RM p287, and p 458).

Hierarchy:

Superclasses: none

Associations:<no rolename> : Unit Of Measure in association UOM**Public Interfaces:Attributes:**
value

Operations: convert

Attributes:

value : Number

The numerical value quantifying the extent or size of some quantity, in the units specified by the associated UnitOfMeasure class.

Operation: convert(target : Unit Of Measure) : Measure

Public member of: Measure

Return Class: Measure

Arguments: Unit Of Measure target

Documentation:

An operation to convert from one unit of measure to another such as feet to meters, or degrees to radians.

7.1.9. Class name: Length

Package the Class belongs to: Unit of Measure

Documentation:The measure of a distance, either directly or as an integral. An example, would be the length of curve, the perimeter of a polygon as the length of the boundary.

Hierarchy:

Superclasses: Measure

Public Interfaces:

Attributes: uom

Attributes:

uom : UomLength

This attribute contains the unit of measure for the Length class, generally expressed as one of the following quantities:

1) meters;

2) feet;

3) kilometers; or

4) miles.

7.1.10. Class name: Angle

Package the Class belongs to: Unit of Measure

Documentation:

The amount of rotation needed to bring one line or plane into coincidence with another, generally measured in degrees, sexagesimal degrees or grads are generally used.

Hierarchy:

Superclasses: Measure

Public Interfaces:

Attributes: uom

Attributes:

uom : UomAngle

This attribute contains the unit of measure for the Angle class, generally expressed as either radians or degrees.

7.1.11. Class name: Area

Package the Class belongs to: Unit of Measure

Documentation:

The measure of the physical extent of any 2-D geometric object.

Hierarchy:

Superclasses: :, Measure

Public Interfaces:

Attributes: uom

Attributes:

uom : UomArea

This attribute contains the unit of measure for the Area class. Common unit of measures include square length units, such as square meters and square feet. Other common units include acres (in the U.S.) and hectares.

7.1.12. Class name: Velocity

Package the Class belongs to: Unit of Measure

Documentation: The measure of motion in terms of speed in a particular direction. It is usually calculated using the simple formula, the change in position during a given time interval.

Hierarchy:

Superclasses: Measure

Public Interfaces:

Attributes: uom

time

distance

Attributes:

time : UomTimeThe designation of a time interval on a selected time scale, astronomical or atomic. It is used in the sense of time of day.distance : UomLength

The extent or amount of space between two objects (e.g., points, lines).

uom : UomLength

This attribute contains the unit of measure for the Velocity class, generally expressed as the change of distance for given time interval (e.g., m/sec).

7.1.13. Class name: Time

Package the Class belongs to: Unit of Measure

Documentation:The designation of an instant on a selected time scale, astronomical or atomic. It is used in the sense of time of day.

Hierarchy:

Superclasses: Measure

Public Interfaces:

Attributes:

uom : UomTime

This attribute contains the unit of measure for the Time class, commonly expressed in one of the following units; seconds, hours or days.

7.1.14. Class name: Scale

Package the Class belongs to: Unit of Measure

Documentation: The ratio of one quantity to another, often unitless.

Hierarchy:

Superclasses: Measure

Public Interfaces:

Attributes: uom

Attributes:

uom : UomScale

This attribute contains the unit of measure for the Scale class (e.g., inches/mile). This class may also not have a unit of measure attribute value, as the scale measure can be unitless.

8. Appendix C. Basic Data Types

This package is another support package for the Coordinate Transformation object model. It contains classes that would probably be used as part of implementing a CT Service.

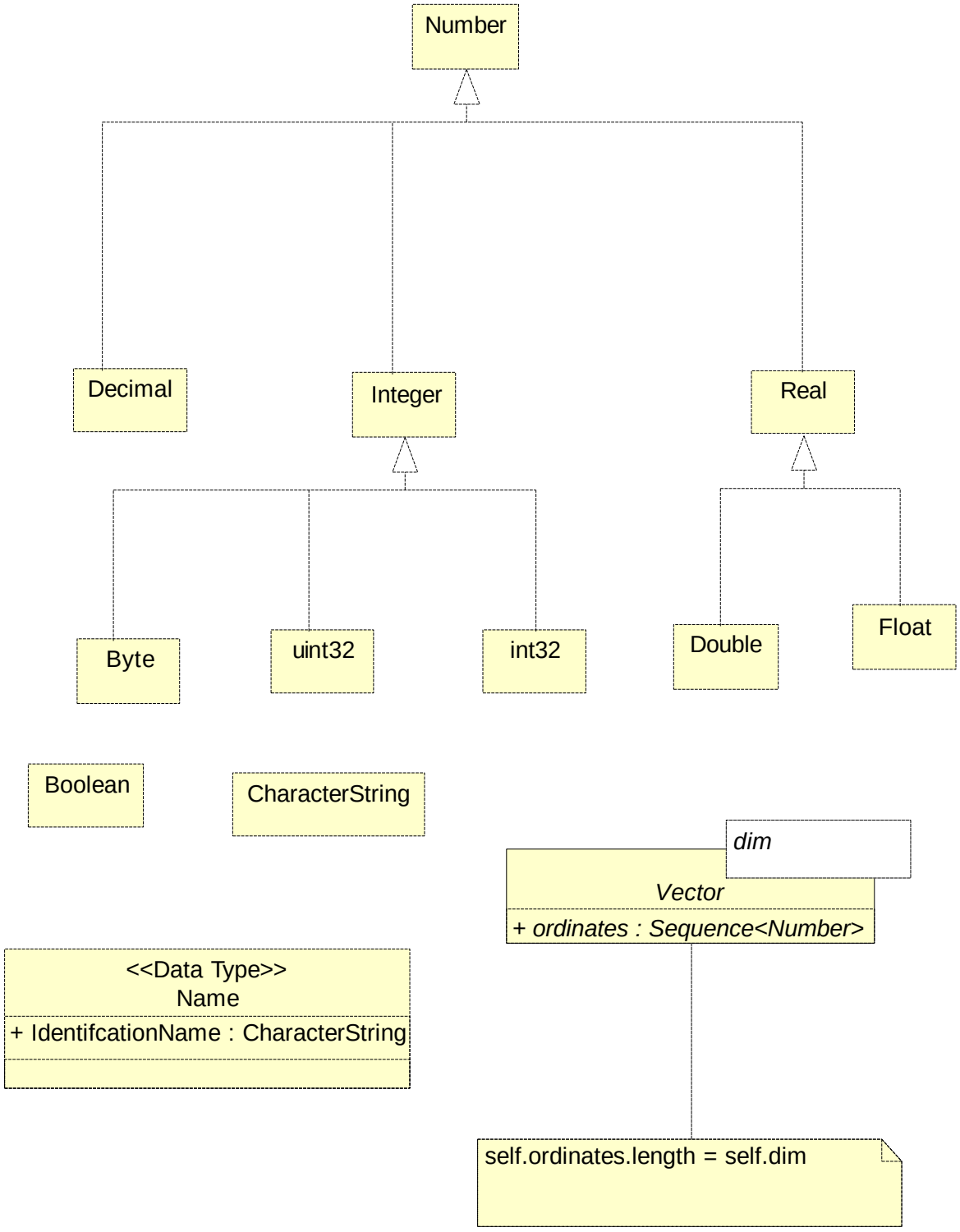

Figure 8-1. Basic Data Types Class Diagram from OGC Geometry Model

8.1.1. Class name: Real

Package the Class belongs to: Basic Data Types

Documentation:

Any number that corresponds to a point on a number line.

Any real can be represented by a (potentially infinite) decimal expansion in any chosen radix. Most computer systems use a radix 2 (binary) expansion of fixed length with an offset exponent.

Hierarchy:

Superclasses: Number

8.1.2. Class name: Float

Package the Class belongs to: Basic Data Types

Documentation:

A limited precision real number used mainly to represent numbers of limited accuracy, such as a surveyed length over long distances.

Usually represented by an IEEE 32 bit real.

Hierarchy:

Superclasses: Real

8.1.3. Class name: Double

Package the Class belongs to: **B**asic Data Types

Documentation:

An extended precision floating point (real) number. Most often, a Double is a 64 bit (8 byte) double precision data type that encodes a double precision number using the IEEE 754 double precision format.

More modern processors, or ones requiring a higher precision could be using a 128 bit floating point. Again, the type is meant to be descriptive as opposed to prescriptive.

Hierarchy:

Superclasses: Real

8.1.4. Class name: int32

Package the Class belongs to: Basic Data Types

Documentation:

An extended precision integer used where +/- 2 billion is necessary and sufficient for a range. Can be used in direct positions where 1 cm accuracy for a worldwide coordinate reference system is sufficient.

A signed 32 bit integer give a range of -2,147,483,647 to + 2,147,483,648.

Hierarchy:

Superclasses: Integer

8.1.5. Class name: uint32

Package the Class belongs to: Basic Data Types

Documentation:

An extended precision unsigned integer used where 0 to 4 billion is necessary and sufficient for a range. It can be used in direct positions where 1 cm accuracy for a world wide coordinate reference system is sufficient.

Normally, an Unsigned Integer (unit) is a 32 bit (4 byte) data type that encodes a non-negative integer in the range of 0 to 4,294,967,295.

Hierarchy:

Superclasses: Integer

8.1.6. Class name: Byte

Package the Class belongs to: Basic Data Types

Documentation:

An integer used to represent for very small whole numbers (i.e., usually an integer less than 64).

Normally, a byte is the smallest directly addressable unit of storage; the amount of memory space used to store one character, or one very small integer is usually 8 bits.

Hierarchy:

Superclasses: Integer

8.1.7. Class name: Boolean

Package the Class belongs to: Basic Data Types

Documentation:

In this context, a binary variable that usually takes a value of zero (FALSE), one (TRUE) or NULL (undecided or unknown).

The value that results from doing a Boolean operation in which each of the operands and the result is a logical truth value.

Hierarchy:

Superclasses: none

8.1.8. Class name: CharacterString

Package the Class belongs to: Basic Data Types

Documentation:

A group or sequence of printable characters, usually from a defined character set or alphabet, such as Roman (Latin-based in common use in Europe) or Cyrillic (Greek based, used in Russian and other Slavic Languages).

A character may be use as a letter, digit, or other symbol that is used to represent information.

Hierarchy:

Superclasses: none

8.1.9. Class name: Vector

Package the Class belongs to: Basic Data Types

Documentation:

A ordered list of numbers, used to represent either a point in a Cartesian coordinate system or a vector (distance and direction), either free floating or attached to a point.

Each number in the list is referred to as an ordinate. The list together as a whole is a coordinate (coordinated set of ordinates).

Hierarchy:

Superclasses: none

Generic parameters:

Integer dim

Public Interfaces:

Attributes: ordinates

Operations: add

multiply

Attributes:

ordinates : Sequence<Number>

An ordered list of numbers

Operations:

add

multiply

8.1.10. Class name: Name

Package the Class belongs to: Basic Data Types

Documentation:

A generic data type class that is currently being used to identify a specify class in a given model being developed.

Hierarchy:

Superclasses: none

Public Interfaces:

Attributes: identificationName

Attributes:

identificationName : CharacterString

A character string used to identify the class (e.g., the name of coordinate transformation or coordinate reference system